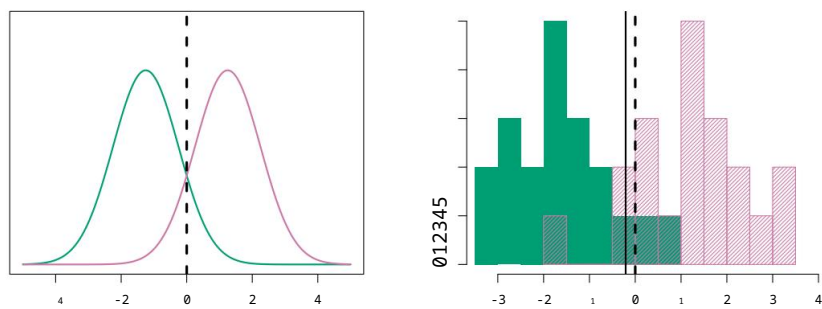




HÌNH 4.4. Bên trái: Hai hàm mật độ phân bố chuẩn một chiều được hiển thị. Đường thẳng đứng đứt đoạn biểu thị ranh giới quyết định Bayes. Bên phải: 20 quan sát được lấy từ mỗi trong hai lớp và được hiển thị dưới dạng biểu đồ cột. Ranh giới quyết định Bayes lại được thể hiện bằng một đường thẳng đứt đoạn. Đường liền nét Đường thẳng đứng biểu thị ranh giới quyết định LDA được ước tính từ quá trình huấn luyện dữ liệu.

$X = x$ đối với lớp mà (4,17) lớn nhất. Lấy logarit của (4,17) và bằng cách sắp xếp lại các điều khoản, không khó để chứng minh rằng điều này tương đương với gán quan sát cho lớp mà

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k) \tag{4.18}$$

là lớn nhất. Ví dụ, nếu $K = 2$ và $\pi_1 = \pi_2$, thì bộ phân loại Bayes gán một quan sát vào lớp 1 nếu $2x(\mu_1 - \mu_2) > \mu_2^2 - \mu_1^2$, và đến lớp 2 trong trường hợp khác. Ranh giới quyết định Bayes là điểm mà tại đó $\delta_1(x) = \delta_2(x)$; người ta có thể chứng minh rằng điều này tương đương với

$$x = \frac{\mu_2^2 - \mu_1^2}{2(\mu_1 - \mu_2)} = \frac{\mu_1 + \mu_2}{2} \tag{4.19}$$

Một ví dụ được minh họa ở bảng bên trái của Hình 4.4. Hai bình thường Các hàm mật độ được hiển thị, $f_1(x)$ và $f_2(x)$, đại diện cho hai giá trị riêng biệt các lớp. Các tham số trung bình và phương sai cho hai hàm mật độ là $\mu_1 = -1,25$, $\mu_2 = 1,25$ và $\sigma_1^2 = \sigma_2^2 = 1$. Hai vùng mật độ chồng lên nhau. và do đó, với giả thiết $X = x$, có một số sự không chắc chắn về lớp mà Quan sát đó thuộc về. Nếu ta giả định rằng một quan sát có khả năng xảy ra như nhau, đến từ một trong hai lớp—tức là $\pi_1 = \pi_2 = 0,5$ —thì bằng cách quan sát (4.19), chúng ta thấy rằng bộ phân loại Bayes gán quan sát vào lớp 1 nếu $x < 0$ và lớp 2 nếu ngược lại. Lưu ý rằng trong trường hợp này, chúng ta có thể tính toán Bộ phân loại Bayes vì chúng ta biết rằng X được lấy từ phân phối Gaussian. sự phân bố trong từng lớp, và chúng ta biết tất cả các tham số liên quan. Trong thực tế, chúng ta không thể tính toán bộ phân loại Bayes.

Trên thực tế, ngay cả khi chúng ta khá chắc chắn về giả định của mình rằng X là được lấy từ phân phối Gaussian trong mỗi lớp, để áp dụng phương pháp Bayes. Đối với bộ phân loại, chúng ta vẫn phải ước tính các tham số $\mu_1, \dots, \mu_K$, $\pi_1, \dots, \pi_K$, và σ^2 . Phương pháp phân tích phân biệt tuyến tính (LDA) xấp xỉ bộ phân loại Bayes bằng cách thay thế các ước tính cho π_k , μ_k và σ^2 vào (4.18). Trong

tuyến tính
phân biệt đối xử
Phân tích

3Xem Bài tập 2 ở cuối chương này.

Cụ thể, các ước tính sau đây được sử dụng:

$$\begin{aligned}\hat{\mu}^k &= \frac{1}{n_k} \sum_{i: y_i=k} x_i \\ \hat{\sigma}^2 &= \frac{1}{n} \sum_{k=1}^K \sum_{i: y_i=k} (x_i - \hat{\mu}^k)^2\end{aligned}\quad (4.20)$$

trong đó n là tổng số quan sát huấn luyện và n_k là số lượng các quan sát huấn luyện trong lớp thứ k . Ước tính cho μ_k đơn giản là trung bình của tất cả các quan sát huấn luyện từ lớp thứ k , trong khi $\hat{\sigma}^2$ có thể được xem như là trung bình có trọng số của các phương sai mẫu cho mỗi K các lớp. Đôi khi chúng ta có kiến thức về xác suất thuộc về lớp $\pi_1, \dots, \pi_K$, có thể được sử dụng trực tiếp. Trong trường hợp không có thêm thông tin nào khác Thông tin này được ước tính bởi LDA dựa trên tỷ lệ các quan sát huấn luyện thuộc về lớp thứ k . Nói cách khác,

$$\hat{\pi}^k = n_k/n. \quad (4.21)$$

Bộ phân loại LDA cầm các ước tính được đưa ra trong (4.20) và (4.21) vào (4.18), và gán một quan sát $X = x$ cho lớp mà

$$\hat{\delta}_k(x) = x \cdot \frac{\hat{\mu}^k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k) \quad (4.22)$$

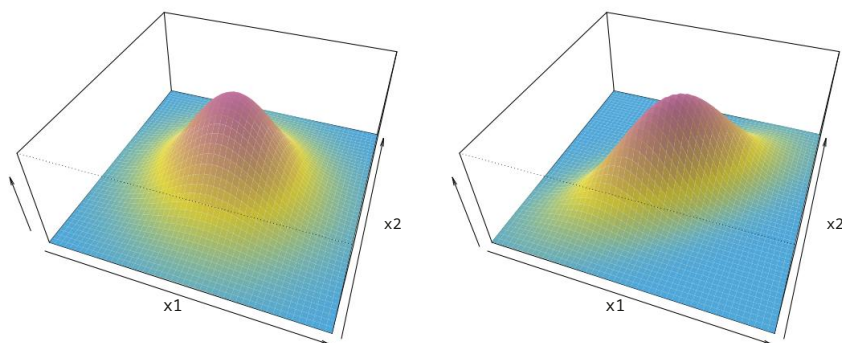
là lớn nhất. Từ "tuyến tính" trong tên của bộ phân loại bắt nguồn từ thực tế rằng các hàm phân biệt $\hat{\delta}_k(x)$ trong (4.22) là các hàm tuyến tính của x (trái ngược với một hàm phức tạp hơn của x).

phân biệt đối xử
chức năng

Bảng bên phải của Hình 4.4 hiển thị biểu đồ tần suất của một mẫu ngẫu nhiên. Mẫu gồm 20 quan sát từ mỗi lớp. Để triển khai LDA, chúng tôi bắt đầu bằng cách ước tính π_k , μ_k và σ^2 bằng cách sử dụng (4.20) và (4.21). Sau đó, chúng tôi đã tính toán ranh giới quyết định, được thể hiện bằng đường kẻ liền màu đen, là kết quả của việc gán một quan sát đối với lớp mà (4.22) là lớn nhất. Tất cả các điểm hướng sang trái phần của đường này sẽ được xếp vào hạng xanh lá cây, trong khi các điểm bên phải của Dòng này được gán cho lớp màu tím. Trong trường hợp này, vì $n_1 = n_2 = 20$, Ta có $\hat{\pi}^1 = \hat{\pi}^2$. Kết quả là, ranh giới quyết định tương ứng với điểm giữa giữa các giá trị trung bình mẫu của hai lớp, $(\hat{\mu}_1 + \hat{\mu}_2)/2$. Hình vẽ cho thấy ranh giới quyết định của LDA nằm hơi lệch về bên trái. ranh giới quyết định Bayes tối ưu, thay vào đó bằng $(\mu_1 + \mu_2)/2 = 0$. Bộ phân loại LDA hoạt động tốt như thế nào trên dữ liệu này? Vì đây là Với dữ liệu mô phỏng, chúng ta có thể tạo ra một số lượng lớn các quan sát thử nghiệm để để tính toán tỷ lệ lỗi Bayes và tỷ lệ lỗi kiểm định LDA. Đó là Lần lượt là 10,6% và 11,1%. Nói cách khác, đó là lỗi của bộ phân loại LDA. Tỷ lệ này chỉ cao hơn 0,5% so với tỷ lệ lỗi nhỏ nhất có thể! Điều này cho thấy rằng LDA đang hoạt động khá tốt trên tập dữ liệu này.

Tóm lại, bộ phân loại LDA xuất phát từ giả định rằng các quan sát trong mỗi lớp đến từ phân phối chuẩn với một lớp-

giá trị trung bình cụ thể và phương sai chung σ^2 , và thay thế các ước tính cho những điều này các tham số vào bộ phân loại Bayes. Trong Phần 4.4.3, chúng ta sẽ xem xét một cách tiếp cận ít chặt chẽ hơn. một tập hợp các giả định nghiêm ngặt, bằng cách cho phép các quan sát trong lớp thứ k . để có phương sai đặc trưng cho từng lớp, σ_k^2 .



HÌNH 4.5. Hai hàm mật độ Gauss đa biến được hiển thị, với $p = 2$. Bên trái: Hai biến dự báo không tương quan. Bên phải: Hai biến có tương quan. Hệ số tương quan là 0,7.

4.4.2 Phân tích phân biệt tuyến tính cho $p > 1$

Giờ đây, chúng ta mở rộng bộ phân loại LDA cho trường hợp có nhiều biến dự đoán. Để làm điều này, chúng ta sẽ giả định rằng $X = (X_1, X_2, \dots, X_p)$ được lấy từ phân phối Gauss đa biến (hoặc phân phối chuẩn đa biến), với vectơ trung bình đặc trưng cho từng lớp và ma trận hiệp phương sai chung. Chúng ta bắt đầu với một phần tóm tắt ngắn gọn về phân phối này.

đa biến
Gaussian

Phân phối Gaussian đa biến giả định rằng mỗi yếu tố dự báo riêng lẻ tuân theo phân phối chuẩn một chiều, như trong (4.16), với một số

Mối tương quan giữa từng cặp biến dự báo. Hai ví dụ về đa biến.

Hình 4.5 hiển thị các phân bố Gaussian với $p = 2$. Chiều cao của

Mặt phẳng tại bất kỳ điểm cụ thể nào biểu thị xác suất cả X_1 và X_2 cùng tồn tại.

và X_2 nằm trong một vùng nhỏ xung quanh điểm đó. Trong cả hai hình, nếu bề mặt được cắt dọc theo trục X_1 hoặc dọc theo trục X_2 , mặt cắt ngang thu được sẽ như thế nào.

sẽ có dạng phân phối chuẩn một chiều. Phía bên trái

Hình 4.5 minh họa một ví dụ trong đó $\text{Var}(X_1) = \text{Var}(X_2)$ và

$\text{Cor}(X_1, X_2) = 0$; bề mặt này có hình dạng chuông đặc trưng. Tuy nhiên,

Hình dạng chuông sẽ bị biến dạng nếu các biến dự báo có tương quan hoặc có phân bố không đồng đều.

sự khác biệt, như được minh họa trong bảng bên phải của Hình 4.5. Trong trường hợp này

Trong trường hợp đó, đáy của quả chuông sẽ có hình elip chứ không phải hình tròn.

hình dạng. Để chỉ ra rằng một biến ngẫu nhiên p chiều X có phân phối Gaussian đa

biến, ta viết $X \sim N(\mu, \Sigma)$. Ở đây $E(X) = \mu$ là

giá trị trung bình của X (một vectơ có p thành phần), và $\text{Cov}(X) = \Sigma$ là

Ma trận hiệp phương sai $p \times p$ của X . Về mặt hình thức, đó là mật độ Gauss đa biến.

được định nghĩa là

$$f(x) = \frac{1}{(2\pi)^p |\Sigma|^{1/2}} \exp\left\{-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right\}. \quad (4.23)$$

Trong trường hợp có $p > 1$ biến dự đoán, bộ phân loại LDA giả định rằng

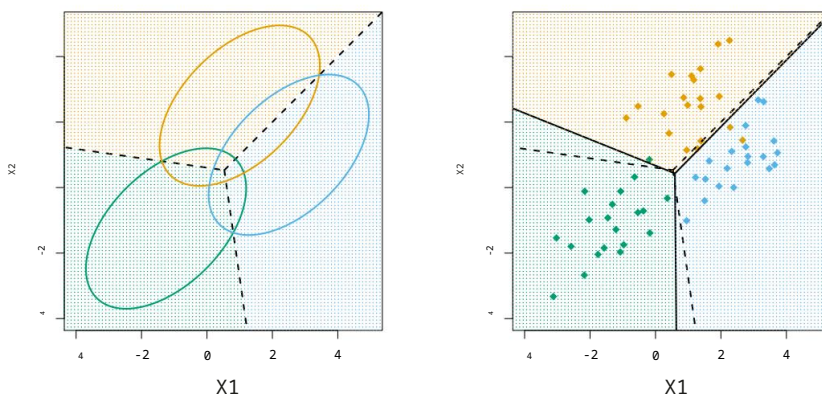
Các quan sát trong lớp thứ k được lấy từ phân phối Gauss đa biến $N(\mu_k, \Sigma)$, trong đó μ_k là

vectơ trung bình đặc trưng cho từng lớp, và Σ là một

ma trận hiệp phương sai chung cho tất cả K lớp. Cắm mật độ vào

hàm cho lớp thứ k , $f_k(X = x)$, vào (4.15) và thực hiện một chút

Một chút đại số cho thấy rằng bộ phân loại Bayes gán cho một quan sát $X = x$



HÌNH 4.6. Một ví dụ với ba lớp. Các quan sát từ mỗi lớp. được lấy từ phân phối Gaussian đa biến với $p = 2$, với vectơ trung bình đặc trưng cho từng lớp và ma trận hiệp phương sai chung. Bên trái: Các hình elip chứa 95 % xác suất cho mỗi trong ba lớp được hiển thị. Các đường nét đứt là các ranh giới quyết định Bayes. Bên phải: 20 quan sát được tạo ra từ mỗi lớp và ranh giới quyết định LDA tương ứng được chỉ ra bằng cách sử dụng Các đường màu đen liền nét. Ranh giới quyết định Bayes một lần nữa được thể hiện bằng các đường nét đứt.

cho lớp học mà

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k \quad (4.24)$$

là lớn nhất. Đây là phiên bản vectơ/ma trận của (4.18).

Một ví dụ được minh họa ở bảng bên trái của Hình 4.6. Ba lớp Gaussian có kích thước bằng nhau được hiển thị với các vectơ trung bình đặc trưng cho từng lớp và một ma trận hiệp phương sai chung. Ba hình elip biểu thị các vùng chứa 95% xác suất cho mỗi trong ba lớp. Các đường nét đứt là các ranh giới quyết định Bayes. Nói cách khác, chúng đại diện cho tập hợp các giá trị x sao cho $\delta_k(x) = \delta(x)$; tức là

$$x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k = x^T \Sigma^{-1} \mu_l - \frac{1}{2} \mu_l^T \Sigma^{-1} \mu_l \quad (4.25)$$

với $k = 1$. (Số hạng $\log \pi_k$ từ (4.24) đã biến mất vì mỗi Ba lớp có cùng số lượng quan sát huấn luyện; tức là π_k là (Giống nhau cho mỗi lớp.) Lưu ý rằng có ba dòng biểu thị Ranh giới quyết định Bayes vì có ba cặp lớp trong số đó. ba lớp. Tức là, một ranh giới quyết định Bayes phân tách lớp 1. Từ lớp 2, người ta tách lớp 1 khỏi lớp 3, và người ta tách lớp 2. từ lớp 3. Ba ranh giới quyết định Bayes này chia bộ dự đoán. chia không gian thành ba vùng. Bộ phân loại Bayes sẽ phân loại một quan sát. tùy thuộc vào khu vực mà nó tọa lạc.

Một lần nữa, chúng ta cần ước lượng các tham số chưa biết $\mu_1, \dots, \mu_K$, $\pi_1, \dots, \pi_K$, và Σ ; các công thức tương tự như các công thức được sử dụng trong trường hợp một chiều, được đưa ra trong (4.20). Để gán một quan sát mới $X = x$, LDA đưa các ước tính này vào (4.24) để thu được các đại lượng $\hat{\delta}_k(x)$ và phân loại vào lớp mà $\hat{\delta}_k(x)$ lớn nhất. Lưu ý rằng trong (4.24) $\delta_k(x)$ là một hàm tuyến tính của x ; nghĩa là, quy tắc quyết định LDA chỉ phụ thuộc vào x .

		Trạng thái mặc định thực sự	
		Không	Có Tổng cộng
Dự đoán	Số	9644	252 9896
Trạng thái mặc định Có		23	81 104
	Tổng cộng	9667 333	10000

BẢNG 4.4. Ma trận nhầm lẫn so sánh các dự đoán của LDA với trạng thái mặc định thực tế cho 10.000 quan sát huấn luyện trong tập dữ liệu **Mặc định** . Các phần tử trên đường chéo của ma trận biểu thị những cá nhân có tình trạng vỡ nợ được dự đoán chính xác, trong khi các phần tử ngoài đường chéo biểu thị những cá nhân bị phân loại sai. LDA đã đưa ra dự đoán sai cho 23 cá nhân không vỡ nợ và cho 252 cá nhân vỡ nợ.

Thông qua sự kết hợp tuyến tính của các phần tử của nó. Như đã thảo luận trước đó, đây là lý do cho từ "tuyến tính" trong LDA.

Trong bảng bên phải của Hình 4.6, 20 quan sát được chọn từ mỗi trong ba lớp được hiển thị, và các ranh giới quyết định LDA thu được được thể hiện bằng các đường màu đen liền nét. Nhìn chung, các ranh giới quyết định LDA khá gần với các ranh giới quyết định Bayes, được thể hiện lại bằng các đường nét đứt.

Tỷ lệ lỗi kiểm thử của bộ phân loại Bayes và LDA lần lượt là 0,0746 và 0,0770. Điều này cho thấy LDA hoạt động tốt trên tập dữ liệu này.

Chúng ta có thể thực hiện LDA trên dữ liệu **Mặc định** để dự đoán liệu một cá nhân có vỡ nợ hay không dựa trên số dư thẻ tín dụng và tình trạng sinh viên . ⁴ Mô hình LDA được huấn luyện trên 10.000 mẫu dữ liệu dẫn đến tỷ lệ lỗi huấn luyện là 2,75%. Con số này nghe có vẻ thấp, nhưng cần lưu ý hai điểm hạn chế.

- Trước hết, tỷ lệ lỗi huấn luyện thường sẽ thấp hơn tỷ lệ lỗi kiểm thử, vốn là đại lượng thực sự cần quan tâm. Nói cách khác, chúng ta có thể dự đoán bộ phân loại này sẽ hoạt động kém hơn nếu sử dụng nó để dự đoán liệu một nhóm cá nhân mới có vỡ nợ hay không. Lý do là chúng ta điều chỉnh các tham số của mô hình một cách cụ thể để hoạt động tốt trên dữ liệu huấn luyện. Tỷ lệ giữa số tham số p và số mẫu n càng cao, thì hiện tượng quá khớp (overfitting) càng đóng vai trò quan trọng. Với dữ liệu này, chúng ta không dự đoán đây sẽ là vấn đề, vì $p = 2$ và $n = 10.000$.

quá khớp

- Thứ hai, vì chỉ có 3,33% số cá nhân trong mẫu huấn luyện vỡ nợ, nên một bộ phân loại đơn giản nhưng vô dụng luôn dự đoán rằng một cá nhân sẽ không vỡ nợ, bất kể số dư thẻ tín dụng và tình trạng sinh viên của họ, sẽ dẫn đến tỷ lệ lỗi là 3,33%. Nói cách khác, bộ phân loại rỗng đơn giản sẽ đạt được tỷ lệ lỗi chỉ cao hơn một chút so với tỷ lệ lỗi của tập huấn luyện LDA.

vô giá trị

Trên thực tế, một bộ phân loại nhị phân như thế này có thể mắc hai loại lỗi: nó có thể gán sai một cá nhân mặc định vào danh mục "không mặc định" , hoặc nó có thể gán sai một cá nhân không mặc định vào...

4. Người đọc kỹ sẽ nhận thấy rằng trạng thái học sinh là định tính – do đó, giả định về phân phối chuẩn được đưa ra bởi LDA rõ ràng bị vi phạm trong ví dụ này! Tuy nhiên, LDA thường rất mạnh mẽ trước các vi phạm mô hình, như ví dụ này cho thấy. Naive Bayes, được thảo luận trong Phần 4.4.4, cung cấp một giải pháp thay thế cho LDA mà không giả định các biến dự báo được phân phối chuẩn.

danh mục mặc định. Việc xác định xem một trong hai danh mục này thuộc loại nào thường rất quan trọng. Các loại lỗi đang xảy ra. Ma trận nhầm lẫn, được hiển thị cho dữ liệu **Mặc định** trong Bảng 4.4, là một cách thuận tiện để hiển thị thông tin này.

lú lẫn
ma trận

Bảng số liệu cho thấy LDA dự đoán tổng cộng 104 người sẽ vỡ nợ. Trong số những người này, 81 người thực sự vỡ nợ và 23 người thì không. Do đó, chỉ có 23 người trong số đó không vỡ nợ. Trong số 9.667 cá nhân không vi phạm hợp đồng, có những người bị dán nhãn sai. Tỷ lệ lỗi này có vẻ khá thấp! Tuy nhiên, trong số 333 cá nhân Mặc định là 252 (hoặc 75,7%) trường hợp đã bị LDA bỏ sót. Vì vậy, trong khi lỗi tổng thể là Tỷ lệ vỡ nợ thấp, nhưng tỷ lệ sai sót trong số những cá nhân vỡ nợ lại rất cao. Từ góc nhìn của một công ty thẻ tín dụng đang cố gắng xác định Đối với những cá nhân có nguy cơ cao, tỷ lệ sai sót là $252/333 = 75,7\%$ trong số các cá nhân. Việc không thực hiện nghĩa vụ có thể là điều không thể chấp nhận được.

Hiệu suất chuyên biệt theo từng lớp cũng rất quan trọng trong y học và sinh học. Trong đó, các thuật ngữ độ nhạy và độ đặc hiệu mô tả hiệu suất của một bộ phân loại hoặc xét nghiệm sàng lọc. Trong trường hợp này, độ nhạy là tỷ lệ phần trăm số người thực sự vỡ nợ được xác định; nó bằng 24,3%. Độ đặc hiệu... là tỷ lệ phần trăm những người không vỡ nợ được xác định chính xác; nó bằng $(1 - 23/9667) = 99,8\%$.

độ nhạy
tính đặc hiệu

Tại sao LDA lại phân loại khách hàng vỡ nợ kém hiệu quả như vậy? Nói cách khác, tại sao nó lại có độ nhạy thấp như vậy? Như chúng ta đã thấy... Như đã thấy, LDA đang cố gắng xấp xỉ bộ phân loại Bayes, vốn có tỷ lệ lỗi tổng thể thấp nhất trong tất cả các bộ phân loại. Nghĩa là, bộ phân loại Bayes sẽ mang lại tổng số quan sát bị phân loại sai ít nhất có thể, bất kể lỗi xuất phát từ lớp nào. Một số trường hợp phân loại sai sẽ Kết quả từ việc gán nhầm khách hàng không mặc định cho Lớp mặc định và các lớp khác sẽ phát sinh do việc gán khách hàng không chính xác. người mặc định thuộc nhóm không mặc định. Ngược lại, một công ty thẻ tín dụng có thể đặc biệt muốn tránh việc phân loại sai một cá nhân sẽ mặc định, trong khi việc phân loại sai một cá nhân sẽ không mặc định, dù vẫn cần tránh, nhưng ít gây vấn đề hơn. Bây giờ chúng ta sẽ thấy rằng Có thể điều chỉnh LDA để phát triển một bộ phân loại đáp ứng tốt hơn các yêu cầu đặt ra. nhu cầu của công ty thẻ tín dụng. Bộ phân loại Bayes hoạt động bằng cách gán một quan sát vào một lớp nhất định. trong đó xác suất hậu nghiệm $p_k(X)$ là lớn nhất. Trong trường hợp hai lớp, điều này tương đương với việc gán một quan sát vào lớp mặc định nếu

$$\Pr(\text{mặc định} = \text{Có} | X = x) > 0,5. \quad (4,26)$$

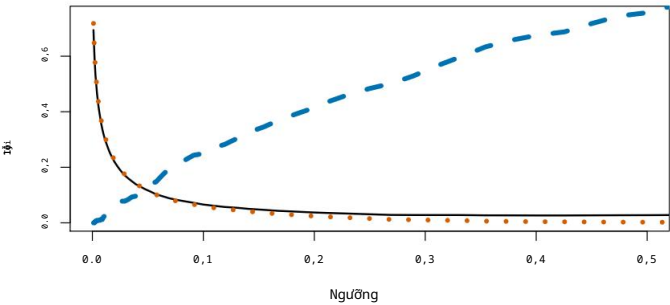
Do đó, bộ phân loại Bayes, và mở rộng ra là LDA, sử dụng ngưỡng 50%. đối với xác suất hậu nghiệm của việc vỡ nợ để gán một quan sát vào lớp mặc định. Tuy nhiên, nếu chúng ta lo ngại về việc dự đoán sai tình trạng mặc định đối với những cá nhân thực sự mặc định, thì chúng ta có thể xem xét hạ thấp ngưỡng này. Ví dụ, chúng ta có thể gán nhãn cho bất kỳ khách hàng nào bằng một Xác suất hậu nghiệm của việc vỡ nợ trên 20% thuộc nhóm vỡ nợ. Nói cách khác, từ ngưỡng, thay vì gán một quan sát cho lớp mặc định nếu (4.26) Nếu điều này đúng, chúng ta có thể gán một quan sát vào lớp này nếu...

$$\Pr(\text{mặc định} = \text{Có} | X = x) > 0,2. \quad (4,27)$$

Tỷ lệ lỗi phát sinh từ phương pháp này được thể hiện trong Bảng 4.5. Hiện tại, LDA dự đoán rằng 430 cá nhân sẽ vỡ nợ. Trong số 333 cá nhân đó, có 333 người. Đối với những trường hợp vỡ nợ, LDA dự đoán chính xác tất cả trừ 138 trường hợp, tương đương 41,4%. Đây là một con số rất lớn.

		Trạng thái mặc định thực sự	
		Không Có	Tổng cộng
Dự đoán	Số 9432	138 9570	
Trạng thái mặc định Có	235 195 430		
	Tổng cộng	9667 333 10000	

BẢNG 4.5. Ma trận nhầm lẫn so sánh các dự đoán của LDA với giá trị thực. Trạng thái mặc định cho 10.000 quan sát huấn luyện trong tập dữ liệu **Mặc định** . sử dụng giá trị ngưỡng được điều chỉnh để dự đoán tình trạng vỡ nợ đối với bất kỳ cá nhân nào có Xác suất vỡ nợ sau đó vượt quá 20 %.



HÌNH 4.7. Đối với bộ dữ liệu **Mặc định** , tỷ lệ lỗi được hiển thị dưới dạng hàm của Giá trị ngưỡng cho xác suất hậu nghiệm được sử dụng để thực hiện việc gán. Đường liền nét màu đen hiển thị tỷ lệ lỗi tổng thể. Đường nét đứt màu xanh lam thể hiện tỷ lệ khách hàng vỡ nợ bị phân loại sai, và Đường chấm màu cam biểu thị tỷ lệ lỗi trong số những người không vi phạm hợp đồng. khách hàng.

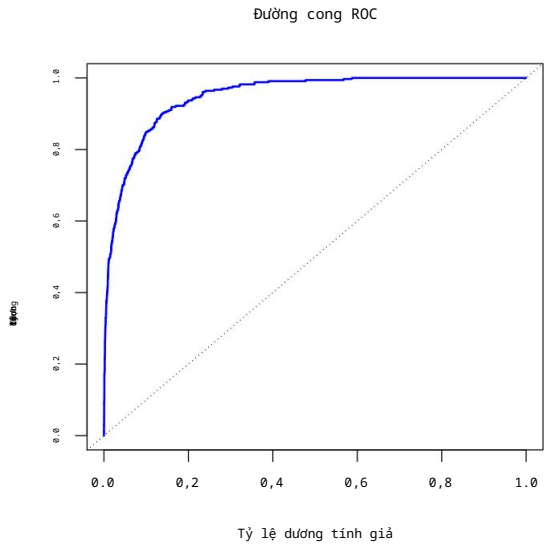
cải thiện tỷ lệ lỗi so với mức 75,7% khi sử dụng phương pháp trước đây. ngưỡng 50%. Tuy nhiên, sự cải thiện này đi kèm với một cái giá: hiện tại là 235 Những cá nhân không vỡ nợ bị phân loại sai. Kết quả là, Tỷ lệ lỗi tổng thể đã tăng nhẹ lên 3,73%. Nhưng một công ty thẻ tín dụng có thể coi sự gia tăng nhẹ về tỷ lệ lỗi tổng thể này là một cái giá nhỏ phải trả. Trả tiền để xác định chính xác hơn những cá nhân thực sự vỡ nợ.

Hình 4.7 minh họa sự đánh đổi phát sinh từ việc điều chỉnh giá trị ngưỡng cho xác suất vỡ nợ hậu nghiệm. Các tỷ lệ lỗi khác nhau là được hiển thị dưới dạng hàm của giá trị ngưỡng. Sử dụng ngưỡng 0,5, như trong (4.26), giảm thiểu tỷ lệ lỗi tổng thể, được thể hiện bằng đường liền nét màu đen. Điều này là điều dễ hiểu, vì bộ phân loại Bayes sử dụng ngưỡng 0,5 và là được biết đến là có tỷ lệ lỗi tổng thể thấp nhất. Nhưng khi ngưỡng là 0,5

Khi sử dụng, tỷ lệ lỗi trong số những người vỡ nợ khá cao (màu xanh). (đường nét đứt). Khi ngưỡng giảm xuống, tỷ lệ lỗi giữa các cá nhân cũng giảm theo. Số người vỡ nợ giảm dần, nhưng tỷ lệ sai sót giữa các cá nhân vẫn cao. Những người không vỡ nợ sẽ tăng lên. Làm thế nào chúng ta có thể quyết định giá trị ngưỡng nào là phù hợp? Tốt nhất? Quyết định như vậy phải dựa trên kiến thức chuyên môn, chẳng hạn như chi tiết. Thông tin về các chi phí phát sinh do vỡ nợ.

Đường cong ROC là một đồ thị phổ biến để hiển thị đồng thời hai loại lỗi cho tất cả các ngưỡng có thể. Tên gọi "ROC" có nguồn gốc lịch sử. và xuất phát từ lý thuyết truyền thông. Nó là từ viết tắt của đặc tính hoạt động của bộ thu (receiver operating characteristics). Hình 4.8 hiển thị đường cong ROC cho LDA. bộ phân loại trên dữ liệu huấn luyện. Hiệu suất tổng thể của bộ phân loại, tổng-

Đường cong ROC



HÌNH 4.8. Đường cong ROC cho bộ phân loại LDA trên tập dữ liệu **Mặc định** .

Nó chỉ ra hai loại lỗi khi chúng ta thay đổi giá trị ngưỡng cho phân bố hậu nghiệm. Xác suất vỡ nợ. Ngưỡng thực tế không được hiển thị. Tỷ lệ dương tính thực sự Độ nhạy là: tỷ lệ người vỡ nợ được xác định chính xác, bằng cách sử dụng một giá trị ngưỡng nhất định. Tỷ lệ dương tính giả là 1 - độ đặc hiệu: phần trăm của những người không vỡ nợ nhưng chúng ta phân loại sai là người vỡ nợ, bằng cách sử dụng cùng ngưỡng đó. giá trị. Đường cong ROC lý tưởng ôm sát góc trên bên trái, cho thấy tỷ lệ dương tính thực cao. tỷ lệ cao và tỷ lệ dương tính giả thấp. Đường chấm chấm biểu thị "không có thông tin". bộ phân loại; đây là điều chúng ta mong đợi nếu xét đến tình trạng sinh viên và số dư thẻ tín dụng. không liên quan đến xác suất vỡ nợ.

được đánh dấu trên tất cả các ngưỡng có thể, được biểu thị bằng diện tích dưới đường cong ROC. đường cong ROC (AUC). Một đường cong ROC lý tưởng sẽ bám sát góc trên bên trái, vì vậy AUC càng lớn thì bộ phân loại càng tốt. Với dữ liệu này, AUC là 0,95, tức là gần mức tối đa 1.0, do đó sẽ được coi là rất tốt. Chúng tôi kỳ vọng một bộ phân loại hoạt động không tốt hơn mức ngẫu nhiên với AUC là 0,5 (khi được đánh giá trên một tập dữ liệu kiểm thử độc lập không được sử dụng trong quá trình huấn luyện mô hình). Đường cong ROC rất hữu ích để so sánh các thuật toán phân loại khác nhau, vì chúng lấy có tính đến tất cả các ngưỡng có thể. Hóa ra đường cong ROC cho Mô hình hồi quy logistic trong Mục 4.3.4 được áp dụng cho dữ liệu này gần như là chính xác. Không thể phân biệt được với mô hình LDA này, vì vậy chúng tôi không hiển thị nó. đây.

Như chúng ta đã thấy ở trên, việc thay đổi ngưỡng phân loại sẽ làm thay đổi giá trị thực của nó. Tỷ lệ dương tính và tỷ lệ dương tính giả. Chúng cũng được gọi là độ nhạy và một trừ đi độ đặc hiệu của bộ phân loại. Vì có rất nhiều thuật ngữ được sử dụng trong ngữ cảnh này, nên chúng tôi sẽ tóm tắt lại. Bảng 4.6

Hiện thị các kết quả có thể xảy ra khi áp dụng bộ phân loại (hoặc xét nghiệm chẩn đoán). đến một quần thể. Để tạo sự liên kết với các tài liệu về dịch tễ học, Chúng ta coi dấu "+" là "bệnh" mà chúng ta đang cố gắng phát hiện, và dấu " " là trạng thái "không bệnh". Để tạo sự liên kết với giả thuyết cổ điển. Trong các tài liệu kiểm chứng, chúng ta coi " " là giả thuyết không và "+" là giả thuyết đúng.

khu vực bên dưới
(ROC)
đường cong

độ nhạy
độ đặc hiệu

BẢNG

		Đang cấp đích thực	
		hoặc Null + hoặc	Tổng khác null
Dự đoán	hoặc Null Âm tính thật (TN)	Âm tính giả (FN) N	
Lớp + hoặc	Không rỗng Vị trí sai (FP)	Vị trí đúng (TP)	P*
Tổng cộng		N	P

BẢNG 4.6. Các kết quả có thể xảy ra khi áp dụng bộ phân loại hoặc xét nghiệm chẩn đoán cho một dân số.

Tên	Sự định nghĩa	Từ đồng nghĩa
Tỷ lệ dương tính giả	Lỗi loại I	FP/N, 1 Độ đặc hiệu
Tỷ lệ dương tính thật	TP/P	1 Lỗi loại II, công suất, độ nhạy, khả năng thu hồi
Giá trị dự đoán dương tính	TP/P	Độ chính xác, 1 tỷ lệ phát hiện sai
tính Giá trị dự đoán âm tính	TN/N*	

BẢNG 4.7. Các biện pháp quan trọng để phân loại và kiểm tra chẩn đoán, được suy ra từ đó. từ các giá trị trong Bảng 4.6.

giả thuyết thay thế (không phải giả thuyết null). Trong ngữ cảnh của dữ liệu **Mặc định** , “+” Dấu “ - ” biểu thị người vi phạm hợp đồng, còn dấu “ + ” biểu thị người không vi phạm.

Bảng 4.7 liệt kê nhiều thước đo hiệu suất phổ biến được sử dụng trong trong ngữ cảnh này. Mẫu số của tỷ lệ dương tính giả và tỷ lệ dương tính thật là số liệu thống kê dân số thực tế trong mỗi lớp. Ngược lại, mẫu số là... Giá trị dự đoán dương và giá trị dự đoán âm là Tổng số lượng dự đoán cho mỗi lớp.

4.4.3 Phân tích phân biệt bậc hai

Như chúng ta đã thảo luận, LDA giả định rằng các quan sát trong mỗi lớp được lấy từ phân phối Gaussian đa biến với đặc điểm cụ thể của từng lớp. vectơ trung bình và ma trận hiệp phương sai chung cho tất cả K lớp.

Phân tích phân biệt bậc hai (QDA) cung cấp một phương pháp tiếp cận thay thế. Giống như LDA, bộ phân loại QDA xuất phát từ giả định rằng các quan sát Các giá trị ước lượng từ mỗi lớp được lấy từ phân phối Gaussian, và các giá trị ước lượng cho các tham số được đưa vào định lý Bayes để thực hiện dự đoán. Tuy nhiên, không giống như LDA, QDA giả định rằng mỗi lớp có các đặc điểm riêng của nó. ma trận hiệp phương sai. Tức là, nó giả định rằng một quan sát từ thứ k lớp có dạng $X \sim N(\mu_k, \Sigma_k)$, trong đó Σ_k là ma trận hiệp phương sai cho lớp thứ k. Dưới giả định này, bộ phân loại Bayes gán một quan sát $X = x$ cho lớp mà

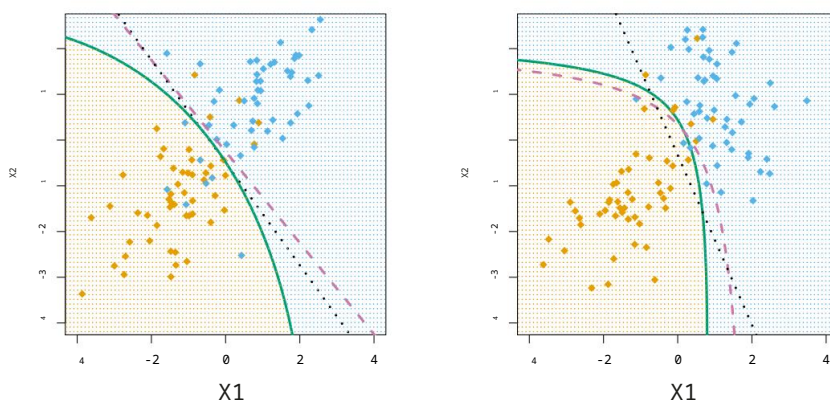
bậc hai
phân biệt đối xử
Phân tích

$$\delta_k(x) = \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) - \frac{1}{2} \log |\Sigma_k| + \log \pi_k$$
$$= \frac{1}{2} x^T \Sigma_k^{-1} x - x^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \log |\Sigma_k| + \log \pi_k$$

(4.28)

là lớn nhất. Vì vậy, bộ phân loại QDA liên quan đến việc tính các ước tính cho Σ_k , μ_k , và π_k vào (4.28), sau đó gán một quan sát $X = x$ cho lớp trong đó đại lượng này lớn nhất. Không giống như trong (4.24), đại lượng x xuất hiện như một hàm bậc hai trong (4.28). Đây là nơi QDA có tên gọi như vậy.

Tại sao việc giả định rằng các lớp K có chung một điểm lại quan trọng? ma trận hiệp phương sai chung? Nói cách khác, tại sao người ta lại ưu tiên LDA hơn



HÌNH 4.9. Bên trái: Đường cong Bayes (đường chấm tím), LDA (đường chấm đen) và QDA. (Đường liền nét màu xanh lá cây) ranh giới quyết định cho bài toán hai lớp với $\Sigma_1 = \Sigma_2$. Vùng tô bóng biểu thị quy tắc quyết định QDA. Vì ranh giới quyết định Bayes là Tuyến tính, nó được xấp xỉ chính xác hơn bằng LDA so với QDA. Bên phải: Chi tiết Các giá trị được trình bày ở bảng bên trái, ngoại trừ $\Sigma_1 = \Sigma_2$. Vì quyết định Bayes Đường biên không phải là tuyến tính, do đó QDA xấp xỉ nó chính xác hơn LDA.

QDA, hay ngược lại? Câu trả lời nằm ở sự đánh đổi giữa độ lệch và phương sai. Khi nào Nếu có p biến dự đoán, thì việc ước tính ma trận hiệp phương sai đòi hỏi phải ước tính $p(p+1)/2$ tham số. QDA ước tính một ma trận hiệp phương sai riêng biệt cho mỗi lớp, với tổng số $Kp(p+1)/2$ tham số. Với 50 biến dự đoán này là bội số của 1.275, tức là rất nhiều tham số. Thay vào đó, bằng cách giả định rằng K lớp có chung ma trận hiệp phương sai, mô hình LDA Nó trở nên tuyến tính theo x , điều này có nghĩa là có Kp hệ số tuyến tính cần ước lượng. Do đó, LDA là một bộ phân loại kém linh hoạt hơn nhiều so với QDA, và do đó có phương sai thấp hơn đáng kể. Điều này có khả năng dẫn đến sự cải thiện hiệu suất dự đoán. Nhưng có một sự đánh đổi: nếu giả định của LDA rằng Ma trận hiệp phương sai chung của K lớp bị sai lệch nghiêm trọng, sau đó LDA được sử dụng có thể bị sai lệch cao. Nói một cách đại khái, LDA thường là lựa chọn tốt hơn so với QDA nếu có tương đối ít quan sát huấn luyện và do đó giảm Độ biến thiên là rất quan trọng. Ngược lại, QDA được khuyến nghị nếu tập huấn luyện là rất lớn, do đó phương sai của bộ phân loại không phải là vấn đề đáng lo ngại, hoặc nếu giả định về ma trận hiệp phương sai chung cho K lớp rõ ràng là

không thể chấp nhận được.

Hình 4.9 minh họa hiệu suất của LDA và QDA trong hai kịch bản khác nhau. Ở bảng bên trái, hai lớp Gaussian có hệ số tương quan chung là 0,7 giữa X_1 và X_2 . Do đó, ranh giới quyết định Bayes có dạng tuyến tính và được xấp xỉ chính xác bởi ranh giới quyết định của LDA. Đường ranh giới quyết định QDA kém hơn vì nó có phương sai cao hơn mà không có sự giảm tương ứng về độ lệch. Ngược lại, phía bên phải Hình minh họa cho thấy tình huống trong đó lớp màu cam có hệ số tương quan là 0,7. Giữa các biến số và lớp màu xanh lam có hệ số tương quan là -0,7. Bây giờ Ranh giới quyết định Bayes là bậc hai, do đó QDA chính xác hơn. Nó xấp xỉ ranh giới này tốt hơn so với LDA.

4.4.4 Thuật toán Naive Bayes

Trong các phần trước, chúng tôi đã sử dụng định lý Bayes (4.15) để phát triển LDA và các bộ phân loại QDA. Ở đây, chúng tôi sử dụng định lý Bayes để giải thích cho các phương pháp phổ biến. Bộ phân loại Naive Bayes .

Bayes ngây thơ

Nhớ lại rằng định lý Bayes (4.15) cung cấp biểu thức cho xác suất hậu nghiệm $p_k(x)$ = $\Pr(Y = k|X = x)$ theo $\pi_1, \dots, \pi_K$ và $f_1(x), \dots, f_K(x)$. Để sử dụng (4.15) trong thực tế, chúng ta cần ước tính cho $\pi_1, \dots, \pi_K$ và $f_1(x), \dots, f_K(x)$. Như chúng ta đã thấy trong các phần trước, việc ước tính phân phối tiên nghiệm Xác suất $\pi_1, \dots, \pi_K$ thường khá đơn giản: ví dụ, chúng ta có thể ước tính π^k là tỷ lệ các quan sát huấn luyện thuộc về nhóm thứ k . lớp, với $k = 1, \dots, K$.

Tuy nhiên, việc ước tính $f_1(x), \dots, f_K(x)$ phức tạp hơn. Nhớ lại rằng $f_k(x)$ là hàm mật độ p chiều cho một quan sát thuộc lớp thứ k . với $k = 1, \dots, K$. Nói chung, việc ước lượng hàm mật độ p chiều là Thách thức. Trong LDA, chúng ta đưa ra một giả định rất mạnh mẽ giúp đơn giản hóa đáng kể nhiệm vụ: chúng ta giả định rằng f_k là hàm mật độ cho một đa biến. biến ngẫu nhiên phân phối chuẩn với giá trị trung bình μ_k đặc trưng cho từng lớp và hiệp phương sai chung ma trận Σ . Ngược lại, trong QDA, chúng ta giả định rằng f_k là hàm mật độ. đối với biến ngẫu nhiên phân phối chuẩn đa biến với giá trị trung bình μ_k đặc trưng cho từng lớp, và ma trận hiệp phương sai đặc trưng cho từng lớp Σ_k . Bằng cách đưa ra những giả định rất mạnh mẽ này, chúng ta có thể thay thế bài toán rất khó khăn là ước lượng K vectơ trung bình các hàm mật độ p chiều với bài toán đơn giản hơn nhiều là ước lượng K vectơ trung bình p chiều và một (trong trường hợp LDA) hoặc K (trong trường hợp của QDA) ma trận hiệp phương sai $(p \times p)$ chiều.

Bộ phân loại Naive Bayes sử dụng một phương pháp khác để ước tính $f_1(x), \dots, f_K(x)$. Thay vì giả định rằng các hàm này thuộc về một nhóm cụ thể nào đó thay vì sử dụng một họ các phân phối (ví dụ: phân phối chuẩn đa biến), chúng ta chỉ tạo ra một phân phối duy nhất. giả định:

Trong lớp thứ k , p biến dự đoán là độc lập.

Nói theo cách toán học, giả định này có nghĩa là đối với $k = 1, \dots, K$,

$$f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \dots \times f_{kp}(x_p), \quad (4.29)$$

trong đó f_{kj} là hàm mật độ của biến dự báo thứ j trong số các quan sát. ở lớp k .

Tại sao giả định này lại có sức mạnh đến vậy? Về cơ bản, việc ước lượng hàm mật độ p chiều rất khó khăn vì chúng ta không chỉ phải xem xét... phân phối biên của mỗi biến dự báo – tức là, phân phối biên mỗi biến dự báo riêng lẻ – nhưng cũng cần xem xét phân phối chung của các biến dự báo. là mối liên hệ giữa các yếu tố dự báo khác nhau. Trong trường hợp của phân phối chuẩn đa biến, mối liên hệ giữa các yếu tố khác nhau Các biến dự báo được tóm tắt bằng các phần tử ngoài đường chéo của ma trận hiệp phương sai. ma trận. Tuy nhiên, nhìn chung, mối liên hệ này rất khó để mô tả và cực kỳ khó ước tính. Nhưng bằng cách giả định rằng Các biến p độc lập trong mỗi lớp, do đó chúng ta loại bỏ hoàn toàn cần phải lo lắng về mối liên hệ giữa các biến dự báo p , bởi vì chúng ta Tôi chỉ đơn giản cho rằng không có mối liên hệ nào giữa các yếu tố dự báo!

Liệu chúng ta có thực sự tin vào giả định Bayes đơn giản rằng p biến độc lập Liệu các thành viên có độc lập trong từng lớp học không? Trong hầu hết các trường hợp, chúng ta không. Nhưng ngay cả Mặc dù giả định mô hình này được đưa ra để thuận tiện, nhưng nó thường dẫn đến...

phân phối – tức chung phân bố

Kết quả khá tốt, đặc biệt trong các trường hợp mà n không đủ lớn so với p để chúng ta có thể ước tính hiệu quả phân phối chung của các biến dự đoán trong mỗi lớp. Trên thực tế, vì việc ước tính phân phối chung đòi hỏi một lượng dữ liệu khổng lồ, nên thuật toán Naive Bayes là một lựa chọn tốt trong nhiều trường hợp. Về cơ bản, giả định của Naive Bayes đưa ra một số sai lệch, nhưng làm giảm phương sai, dẫn đến một bộ phân loại hoạt động khá tốt trong thực tế nhờ sự đánh đổi giữa sai lệch và phương sai.

Sau khi đưa ra giả định Bayes đơn giản, chúng ta có thể thay (4.29) vào (4.15) để có được biểu thức cho xác suất hậu nghiệm,

$$\Pr(Y = k | X = x) = \frac{\pi_k \times \prod_{j=1}^K f_{kj}(x_j)}{\sum_{l=1}^K \pi_l \times \prod_{j=1}^K f_{lj}(x_j)} \quad (4.30)$$

với $k = 1, \dots, K$.

Để ước tính hàm mật độ một chiều f_{kj} bằng cách sử dụng dữ liệu huấn luyện, chúng ta có một vài lựa chọn. $x_{1j}, \dots, x_{nj}$,

- Nếu X_j là biến định lượng, thì ta có thể giả định rằng $X_j | Y = k \sim N(\mu_{jk}, \sigma^2_{jk})$. Nói cách khác, chúng ta giả định rằng trong mỗi lớp, biến dự báo thứ j được lấy từ phân phối chuẩn (đơn biến). Mặc dù điều này nghe có vẻ hơi giống QDA, nhưng có một điểm khác biệt quan trọng, đó là ở đây chúng ta giả định rằng các biến dự báo là độc lập; điều này tương đương với QDA với một giả định bổ sung rằng ma trận hiệp phương sai đặc trưng cho từng lớp là ma trận đường chéo.

- Nếu X_j là biến định lượng, thì một lựa chọn khác là sử dụng ước lượng phi tham số cho f_{kj} . Một cách rất đơn giản để làm điều này là tạo biểu đồ tần suất cho các quan sát của biến dự báo thứ j trong mỗi lớp.

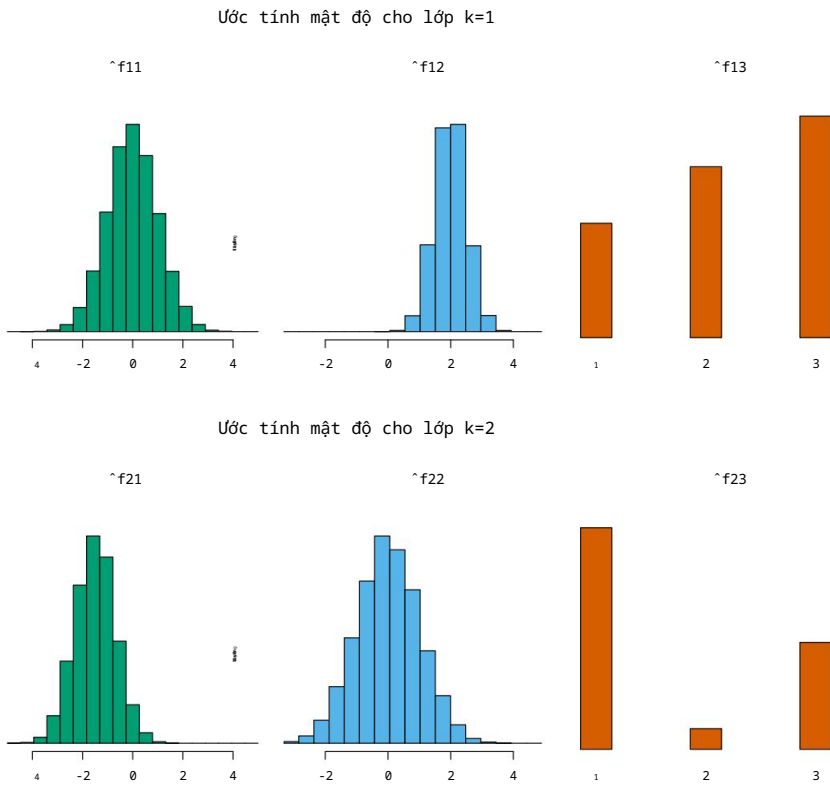
Sau đó, chúng ta có thể ước tính $f_{kj}(x_j)$ là tỷ lệ các quan sát huấn luyện trong lớp thứ k thuộc cùng một ô tần số với x_j . Ngoài ra, chúng ta có thể sử dụng bộ ước lượng mật độ hạt nhân, về cơ bản là một phiên bản được làm mịn của biểu đồ tần số.

ước
lượng
mật độ hạt nhân

- Nếu X_j là biến định tính, thì ta có thể đơn giản đếm tỷ lệ các quan sát huấn luyện cho bộ dự đoán thứ j tương ứng với mỗi lớp. Ví dụ, giả sử $X_j \in \{1, 2, 3\}$, và ta có 100 quan sát trong lớp thứ k . Giả sử bộ dự đoán thứ j nhận các giá trị 1, 2 và 3 trong lần lượt 32, 55 và 13 quan sát đó. Khi đó, ta có thể ước tính f_{kj} như sau:

$$\hat{f}_{kj}(x_j) = \begin{cases} 0,32 & \text{nếu } x_j = 1 \\ 0,55 & \text{nếu } x_j = 2 \\ 0,13 & \text{nếu } x_j = 3. \end{cases}$$

Bây giờ chúng ta xem xét bộ phân loại Naive Bayes trong một ví dụ đơn giản với $p = 3$ biến dự đoán và $K = 2$ lớp. Hai biến dự đoán đầu tiên là định lượng, và biến dự đoán thứ ba là định tính với ba mức. Giả sử thêm rằng $\pi^1 = \pi^2 = 0,5$. Các hàm mật độ ước tính $\hat{f}_{kj}$ cho $k = 1, 2$ và $j = 1, 2, 3$ được hiển thị trong Hình 4.10. Bây giờ giả sử chúng ta muốn phân loại một quan sát mới, $x = (0,4, 1,5, 1)^T$. Hóa ra trong trường hợp này



HÌNH 4.10. Trong ví dụ đồ chơi ở Mục 4.4.4, chúng ta tạo dữ liệu với $p = 3$ các biến dự báo và $K = 2$ lớp. Hai biến dự báo đầu tiên là định lượng, và Biến dự báo thứ ba là biến định tính với ba cấp độ. Trong mỗi lớp, mật độ ước tính. Kết quả cho mỗi trong ba biến dự đoán được hiển thị. Nếu xác suất tiên nghiệm cho hai nếu các lớp bằng nhau thì quan sát $x = (0,4, 1,5, 1)$ có xác suất hậu nghiệm là 94,4%. xác suất thuộc về lớp đầu tiên.

		Trạng thái mặc định thực sự	
		Không Có	Tổng cộng
Dự đoán	Số	9621	244 9865
Trạng thái mặc định Có		46	89
	Tổng cộng	9667	333 10000

BẢNG 4.8. So sánh dự đoán của thuật toán Bayes đơn giản với giá trị mặc định thực tế trạng thái cho 10.000 quan sát huấn luyện trong tập dữ liệu Mặc định, khi chúng ta Dự đoán tình trạng vỡ nợ đối với bất kỳ quan sát nào mà $P(Y = \text{vỡ nợ} | X = x) > 0,5$.

ví dụ, $\hat{f}_{11}(0.4) = 0.368$, $\hat{f}_{12}(1.5) = 0.484$, $\hat{f}_{13}(1) = 0.226$ và $\hat{f}_{21}(0.4) = 0.030$, $\hat{f}_{22}(1,5) = 0.130$, $\hat{f}_{23}(1) = 0.616$. Thay các ước tính này vào (4,30) kết quả là ước tính xác suất hậu nghiệm của $\Pr(Y = 1 | X = x) = 0,944$ và $\Pr(Y = 2 | X = x) = 0,056$.

Bảng 4.8 cung cấp ma trận nhầm lẫn thu được từ việc áp dụng phương pháp đơn giản. Áp dụng bộ phân loại Bayes cho tập dữ liệu Mặc định, trong đó chúng ta dự đoán tình trạng vỡ nợ nếu... Xác suất hậu nghiệm của việc vỡ nợ – tức là $P(Y = \text{vỡ nợ} | X = x)$ – vượt quá 0,5. So sánh điều này với kết quả của LDA trong Bảng 4.4, phát hiện của chúng tôi cho thấy... Kết quả khá trái chiều. Mặc dù LDA có tỷ lệ lỗi tổng thể thấp hơn một chút, nhưng Naive Bayes...

4.5 So sánh các phương pháp phân loại 161

		Trạng thái mặc định thực sự	
		Không	Có
Dự đoán	Số	9339	130 9469
	Trạng thái mặc định	Có 328 203	531
Tổng cộng		9667 333	10000

BẢNG 4.9. So sánh các dự đoán Bayes đơn giản với trạng thái mặc định thực sự cho 10.000 quan sát huấn luyện trong tập dữ liệu **Mặc định**, khi chúng ta dự đoán mặc định cho bất kỳ quan sát nào mà $P(Y = \text{mặc định} | X = x) > 0,2$.

Phương pháp này dự đoán chính xác hơn tỷ lệ người vỡ nợ thực sự. Trong cách triển khai thuật toán Naive Bayes này, chúng tôi giả định rằng mỗi biến dự báo định lượng được lấy từ phân phối Gaussian (và tất nhiên, trong mỗi lớp, mỗi biến dự báo là độc lập).

Cũng giống như với LDA, chúng ta có thể dễ dàng điều chỉnh ngưỡng xác suất để dự đoán vỡ nợ. Ví dụ, Bảng 4.9 cung cấp ma trận nhầm lẫn thu được từ việc dự đoán vỡ nợ nếu $P(Y = \text{vỡ nợ} | X = x) > 0,2$. Một lần nữa, kết quả khá hỗn hợp so với LDA với cùng ngưỡng (Bảng 4.5).

Thuật toán Naive Bayes có tỷ lệ lỗi cao hơn, nhưng dự đoán chính xác gần hai phần ba số trường hợp vỡ nợ thực tế.

Trong ví dụ này, không có gì đáng ngạc nhiên khi thuật toán Naive Bayes không cho kết quả vượt trội hơn hẳn so với LDA: tập dữ liệu này có $n = 10.000$ và $p = 2$, do đó việc giảm phương sai do giả định của Naive Bayes mang lại không nhất thiết là đáng giá. Chúng ta kỳ vọng sẽ thấy lợi ích lớn hơn khi sử dụng Naive Bayes so với LDA hoặc QDA trong các trường hợp p lớn hơn hoặc n nhỏ hơn, khi đó việc giảm phương sai là rất quan trọng.

4.5 So sánh các phương pháp phân loại

4.5.1 So sánh phân tích Bây giờ chúng ta

sẽ thực hiện so sánh phân tích (hoặc toán học) giữa LDA, QDA, Naive Bayes và hồi quy logistic. Chúng ta xem xét các phương pháp này trong một thiết lập với K lớp, sao cho chúng ta gán một quan sát vào lớp tối đa hóa $\Pr(Y = k | X = x)$. Tương đương, chúng ta có thể đặt K là lớp cơ sở và gán một quan sát vào lớp tối đa hóa $\Pr(Y = k | X = x)$.

nhật ký

$$\frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)}$$

(4.31)

với $k = 1, \dots, K$. Việc xem xét dạng cụ thể của (4.31) cho mỗi phương pháp sẽ giúp hiểu rõ hơn về sự giống nhau và khác nhau của chúng.

Đầu tiên, đối với LDA, chúng ta có thể sử dụng định lý Bayes (4.15) cũng như giả định rằng các biến dự đoán trong mỗi lớp được lấy từ mật độ chuẩn đa biến (4.23) với giá trị trung bình cụ thể của lớp và đồng biến chung.

ma trận phương sai để chứng minh rằng

$$\begin{aligned}
 \text{nhật ký} \quad \frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)} &= \log \frac{\pi_k f_k(x)}{\pi_K f_K(x)} \\
 &= \log \frac{\pi_k \exp \left(-\frac{1}{2} (x - \mu_k)^T \Sigma^{-1} (x - \mu_k) \right)}{\pi_K \exp \left(-\frac{1}{2} (x - \mu_K)^T \Sigma^{-1} (x - \mu_K) \right)} \\
 &= \log \frac{\pi_k}{\pi_K} - \frac{1}{2} (x - \mu_k)^T \Sigma^{-1} (x - \mu_k) \\
 &\quad + \frac{1}{2} (x - \mu_K)^T \Sigma^{-1} (x - \mu_K) \\
 &= \log \frac{\pi_k}{\pi_K} - \frac{1}{2} (\mu_k + \mu_K)^T \Sigma^{-1} (\mu_k - \mu_K) \\
 &\quad + x^T \Sigma^{-1} (\mu_k - \mu_K) \\
 &= a_k + \sum_{j=1}^p b_{kj} x_j, \tag{4.32}
 \end{aligned}$$

trong đó $a_k = \frac{\pi_k}{\pi_K} - \frac{1}{2} (\mu_k + \mu_K)^T \Sigma^{-1} (\mu_k - \mu_K)$ và b_{kj} là thứ j

thành phần logarit của $\Sigma^{-1} (\mu_k - \mu_K)$. Do đó, LDA, giống như hồi quy logistic, giả định logarit của xác suất hậu nghiệm là tuyến tính theo x .

Sử dụng các phép tính tương tự, trong thiết lập QDA (4.31) trở thành

$$\text{nhật ký} \quad \frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)} = a_k + \sum_{j=1}^p b_{kj} x_j + \sum_{j=1}^p \sum_{l=1}^p c_{kjl} x_j x_l, \tag{4.33}$$

trong đó a_k , b_{kj} và c_{kjl} là các hàm của π_k , π_K , μ_k , μ_K , Σ_k và Σ_K . Một lần nữa, Như tên gọi cho thấy, QDA giả định rằng logarit tỷ lệ xác suất hậu nghiệm Xác suất tỷ lệ thuận với bình phương của x .

Cuối cùng, chúng ta xem xét (4.31) trong thiết lập Bayes đơn giản. Nhớ lại rằng trong Trong thiết lập này, $f_k(x)$ được mô hình hóa như là tích của p hàm một chiều.

$f_{kj}(x_j)$ cho $j = 1, \dots, p$. Do đó,

$$\begin{aligned}
 \text{nhật ký} \quad \frac{\Pr(Y = k|X = x)}{\Pr(Y = K|X = x)} &= \log \frac{\pi_k f_k(x)}{\pi_K f_K(x)} \\
 &= \log \frac{\pi_k \prod_{j=1}^p f_{kj}(x_j)}{\pi_K \prod_{j=1}^p f_{Kj}(x_j)} \\
 &= \log \frac{\pi_k}{\pi_K} + \sum_{j=1}^p \text{nhật ký} \frac{f_{kj}(x_j)}{f_{Kj}(x_j)} \\
 &= a_k + \sum_{j=1}^p g_{kj}(x_j), \tag{4.34}
 \end{aligned}$$

trong đó $a_k = \log \frac{\pi_k}{\pi_K}$ và $g_{kj}(x_j) = \log \frac{f_{kj}(x_j)}{f_{Kj}(x_j)}$. Do đó, về phía

Mặt (4.34) có dạng mô hình cộng tính tổng quát, một chủ đề là

Sẽ được thảo luận chi tiết hơn trong Chương 7.

4.5 So sánh các phương pháp phân loại 163

Kiểm tra (4.32), (4.33) và (4.34) thu được các quan sát sau

Về LDA, QDA và Naive Bayes:

- LDA là trường hợp đặc biệt của QDA với $\text{ckjl} = 0$ cho tất cả $j = 1, \dots, p$, $l = 1, \dots, p$ và $k = 1, \dots, K$. (Tất nhiên, điều này không có gì đáng ngạc nhiên, vì LDA chỉ đơn giản là một phiên bản hạn chế của QDA với $\Sigma_1 = \dots = \Sigma_K = \Sigma$.)
- Bất kỳ bộ phân loại nào có ranh giới quyết định tuyến tính đều là trường hợp đặc biệt của Naive Bayes với $g_{kj}(x_j) = b_{kj}x_j$. Cụ thể, điều này có nghĩa là LDA là trường hợp đặc biệt của Naive Bayes! Điều này hoàn toàn không rõ ràng từ các mô tả về LDA và Naive Bayes trước đó trong chương này, vì mỗi phương pháp đưa ra những giả định rất khác nhau: LDA giả định rằng các đặc trưng được phân bố chuẩn với ma trận hiệp phương sai chung trong lớp, và Naive Bayes thì giả định sự độc lập của các đặc trưng.
- Nếu chúng ta mô hình hóa $f_{kj}(x_j)$ trong bộ phân loại Bayes đơn giản bằng cách sử dụng phân phối Gaussian một chiều $N(\mu_{kj}, \sigma^2)$, thì ta sẽ có $g_{kj}(x_j) = \frac{1}{\sigma^2} b_{kj}x_j$ trong đó $b_{kj} = (\mu_{kj} - \mu_K) / \sigma^2$. Trong trường hợp này, Bayes đơn giản thực chất là một trường hợp đặc biệt của LDA với Σ bị giới hạn là một ma trận đường chéo với phần tử đường chéo thứ j bằng σ^2 .
- Cả QDA và Naive Bayes đều không phải là trường hợp đặc biệt của nhau. Naive Bayes có thể tạo ra sự phù hợp linh hoạt hơn, vì có thể chọn bất kỳ giá trị nào cho $g_{kj}(x_j)$. Tuy nhiên, nó bị hạn chế ở sự phù hợp thuần túy cộng tính, theo nghĩa là trong (4.34), một hàm của x_j được cộng với một hàm của x_l , với $j = 1$; tuy nhiên, các thuật ngữ này không bao giờ được nhân với nhau. Ngược lại, QDA bao gồm các thuật ngữ nhân có dạng $g_{kjlx}x_l$. Do đó, QDA có tiềm năng chính xác hơn trong các trường hợp mà sự tương tác giữa các biến dự đoán là quan trọng trong việc phân biệt giữa các lớp.

Không có phương pháp nào vượt trội hơn hẳn các phương pháp khác: trong bất kỳ trường hợp nào, việc lựa chọn phương pháp sẽ phụ thuộc vào phân bố thực sự của các biến dự báo trong mỗi lớp K , cũng như các yếu tố khác, chẳng hạn như giá trị của n và p . Yếu tố sau liên quan đến sự đánh đổi giữa độ lệch và phương sai.

Hồi quy logistic liên quan đến câu chuyện này như thế nào? Hãy nhớ lại từ (4.12) rằng Hồi quy logistic đa biến có dạng

$$\text{nhật ký} \quad \frac{\Pr(Y = k | X = x)}{\Pr(Y = K | X = x)} = \beta_{k0} + \sum_{j=1}^p \beta_{kj} x_j.$$

Điều này giống hệt với dạng tuyến tính của LDA (4.32): trong cả hai trường hợp, logarit của $\Pr(Y = k | X = x)$ là một hàm tuyến tính của các biến dự đoán. Trong LDA, các hệ số trong hàm tuyến tính này là các hàm của các ước tính cho π_K , μ_K , μ_k và Σ thu được bằng cách giả định rằng $X_1, \dots, X_p$ tuân theo phân phối chuẩn trong mỗi lớp.

Ngược lại, trong hồi quy logistic, các hệ số được chọn để tối đa hóa hàm khả năng (4.5). Do đó, chúng ta kỳ vọng LDA sẽ hoạt động tốt hơn hồi quy logistic khi giả định về phân phối chuẩn (xấp xỉ) đúng, và chúng ta kỳ vọng hồi quy logistic sẽ hoạt động tốt hơn khi giả định này không đúng.

164 4. Phân loại

Chúng ta sẽ kết thúc bằng một cuộc thảo luận ngắn gọn về thuật toán K-nearest neighbors (KNN), được giới thiệu trong Chương 2. Hãy nhớ rằng KNN sử dụng một phương pháp hoàn toàn khác so với các thuật toán phân loại đã được trình bày trong chương này. Để đưa ra dự đoán cho một quan sát $X = x$, các quan sát huấn luyện gần nhất với x sẽ được xác định. Sau đó, X được gán vào lớp mà phần lớn các quan sát này thuộc về. Do đó, KNN là một phương pháp hoàn toàn phi tham số: không có giả định nào được đưa ra về hình dạng của ranh giới quyết định. Chúng ta sẽ đưa ra những nhận xét sau về KNN:

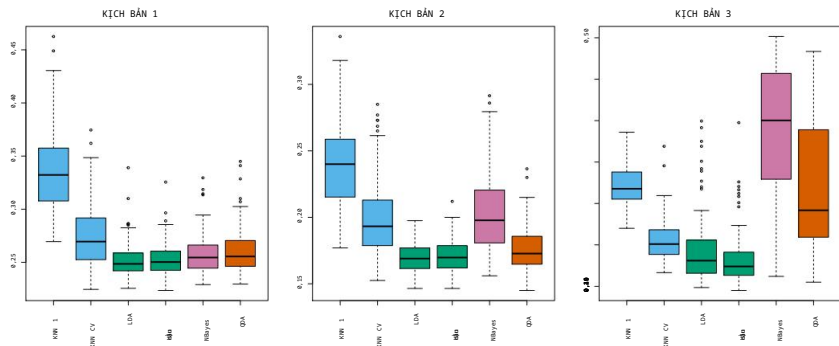
- Vì KNN hoàn toàn không tham số, chúng ta có thể kỳ vọng phương pháp này sẽ vượt trội hơn LDA và hồi quy logistic khi ranh giới quyết định có tính phi tuyến cao, với điều kiện n rất lớn và p nhỏ.
- Để phân loại chính xác, thuật toán KNN yêu cầu số lượng quan sát lớn so với số lượng biến dự đoán-nghĩa là n lớn hơn nhiều so với p . Điều này liên quan đến việc KNN là thuật toán phi tham số, do đó có xu hướng giảm độ lệch trong khi vẫn tạo ra nhiều biến thiên.
- Trong các trường hợp ranh giới quyết định không tuyến tính nhưng n chỉ ở mức vừa phải, hoặc p không quá nhỏ, thì QDA có thể được ưu tiên hơn KNN. Điều này là do QDA có thể cung cấp ranh giới quyết định phi tuyến tính trong khi tận dụng dạng tham số, có nghĩa là nó yêu cầu kích thước mẫu nhỏ hơn để phân loại chính xác, so với KNN.
- Khác với hồi quy logistic, KNN không cho chúng ta biết biến dự báo nào là quan trọng: chúng ta không nhận được bảng hệ số như trong Bảng 4.3.

4.5.2 So sánh thực nghiệm

Chúng ta sẽ so sánh hiệu suất thực nghiệm (thực tế) của hồi quy logistic, LDA, QDA, Naive Bayes và KNN. Chúng tôi đã tạo dữ liệu từ sáu kịch bản khác nhau, mỗi kịch bản liên quan đến một bài toán phân loại nhị phân (hai lớp). Trong ba kịch bản, ranh giới quyết định của Bayes là tuyến tính, và trong các kịch bản còn lại, nó là phi tuyến tính. Đối với mỗi kịch bản, chúng tôi đã tạo ra 100 tập dữ liệu huấn luyện ngẫu nhiên. Trên mỗi tập dữ liệu huấn luyện này, chúng tôi đã áp dụng từng phương pháp vào dữ liệu và tính toán tỷ lệ lỗi kiểm thử thu được trên một tập dữ liệu kiểm thử lớn. Kết quả cho các kịch bản tuyến tính được hiển thị trong Hình 4.11, và kết quả cho các kịch bản phi tuyến tính được hiển thị trong Hình 4.12. Phương pháp KNN yêu cầu lựa chọn K , số lượng láng giềng (không nên nhầm lẫn với số lượng lớp trong các phần trước của chương này). Chúng tôi đã thực hiện thuật toán KNN với hai giá trị của K : $K = 1$, và một giá trị K được chọn tự động bằng phương pháp kiểm định chéo, mà chúng tôi sẽ thảo luận chi tiết hơn trong Chương 5. Chúng tôi đã áp dụng thuật toán Bayes đơn giản với giả định mật độ Gaussian đơn biến cho các đặc trưng trong mỗi lớp (và, tất nhiên - vì đây là đặc điểm chính của Bayes đơn giản - giả định tính độc lập của các đặc trưng).

Trong mỗi sáu kịch bản, đều có $p = 2$ biến dự báo định lượng. Các tình huống cụ thể như sau:

4.5 So sánh các phương pháp phân loại 165



HÌNH 4.11. Biểu đồ hộp thể hiện tỷ lệ lỗi kiểm thử cho từng kịch bản tuyến tính. được mô tả trong văn bản chính.

Tình huống 1: Có 20 lượt quan sát huấn luyện trong mỗi lớp học, chia thành hai lớp. Các quan sát trong mỗi lớp là các biến ngẫu nhiên phân phối chuẩn không tương quan.

với giá trị trung bình khác nhau trong mỗi lớp. Bảng bên trái của Hình 4.11 cho thấy LDA đã thể hiện tốt trong điều kiện này, đúng như mong đợi vì đây là Mô hình được giả định bởi LDA. Hồi quy logistic cũng cho kết quả khá tốt.

Vì nó giả định một ranh giới quyết định tuyến tính. KNN hoạt động kém hiệu quả vì Nó phải trả giá bằng sự biến động mà không được bù đắp bằng việc giảm thiểu sai lệch. QDA cũng cho kết quả kém hơn LDA, vì nó phù hợp với một bộ phân loại linh hoạt hơn. hơn mức cần thiết. Hiệu suất của thuật toán Naive Bayes tốt hơn một chút so với... QDA, bởi vì giả định đơn giản của phương pháp Bayes về các biến dự báo độc lập là Chính xác.

Kịch bản 2: Chỉ tiết tương tự như Kịch bản 1, ngoại trừ việc trong mỗi lớp, thì...

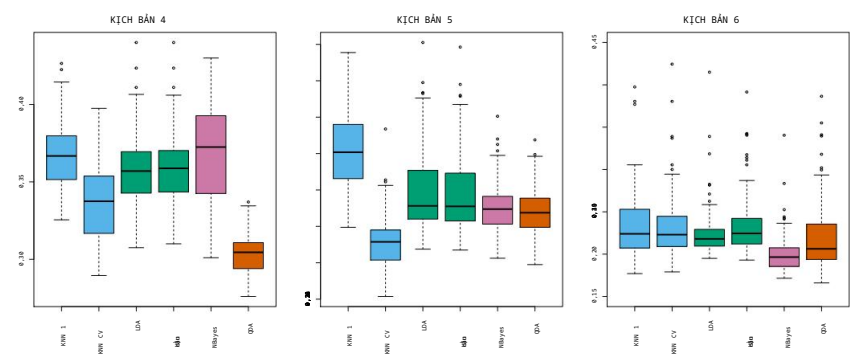
Hai biến dự báo có hệ số tương quan là -0,5. Bảng giữa của Hình 4.11 Điều này cho thấy hiệu suất của hầu hết các phương pháp đều tương tự như trường hợp trước đó. Ngoại lệ đáng chú ý là Naive Bayes, phương pháp này hoạt động rất tốt. Điều này không tốt ở đây, vì giả định Bayes đơn giản về các biến dự đoán độc lập là bị vi phạm.

Kịch bản 3: Cũng như kịch bản trước, có sự tương quan nghịch đáng kể giữa các biến dự báo trong mỗi lớp. Tuy nhiên, lần này chúng ta

Đã tạo X1 và X2 từ phân phối t, với 50 quan sát cho mỗi lớp. Phân phối t có hình dạng tương tự như phân phối chuẩn, nhưng nó... Có xu hướng tạo ra nhiều điểm cực đoan hơn-nghĩa là, nhiều điểm hơn cách xa giá trị trung bình. Trong trường hợp này, ranh giới quyết định vẫn là tuyến tính. và do đó phù hợp với khuôn khổ hồi quy logistic. Thiết lập này đã vi phạm giả định của LDA, vì các quan sát không được lấy từ một mẫu chuẩn. phân phối. Bảng bên phải của Hình 4.11 cho thấy hồi quy logistic hoạt động tốt hơn LDA, mặc dù cả hai phương pháp đều vượt trội hơn phương pháp còn lại. các phương pháp tiếp cận. Đặc biệt, kết quả QDA giảm sút đáng kể khi... Hậu quả của việc không tuân theo phân phối chuẩn. Thuật toán Naive Bayes hoạt động rất kém vì... Giả định về tính độc lập đã bị vi phạm.

Kịch bản 4: Dữ liệu được tạo ra từ phân phối chuẩn, với một hệ số tương quan là 0,5 giữa các biến dự báo trong lớp đầu tiên, và hệ số tương quan của -0,5 giữa các biến dự đoán trong lớp thứ hai. Cấu hình này tương ứng với giả định QDA, và dẫn đến các ranh giới quyết định bậc hai. Hình 4.12 , bảng bên trái, cho thấy QDA vượt trội hơn tất cả các phương pháp khác.

t-
phân bố



HÌNH 4.12. Biểu đồ hộp của tỷ lệ lỗi kiểm tra cho mỗi phương pháp phi tuyến tính. các kịch bản được mô tả trong văn bản chính.

các phương pháp tiếp cận khác. Giả định Bayes đơn giản về các yếu tố dự báo độc lập.

Điều này bị vi phạm, do đó thuật toán Naive Bayes hoạt động kém hiệu quả.

Kịch bản 5: Dữ liệu được tạo ra từ phân phối chuẩn với các biến dự báo không tương quan.

Sau đó, các phản hồi được lấy mẫu từ phân phối logistic.

hàm được áp dụng cho một hàm phi tuyến phức tạp của các biến dự báo.

Bảng giữa của Hình 4.12 cho thấy cả QDA và Naive Bayes đều cho kết quả như vậy.

cho kết quả tốt hơn một chút so với các phương pháp tuyến tính, trong khi phương pháp KNN-CV linh hoạt hơn nhiều lại cho kết quả tốt nhất. Nhưng KNN với K = 1 lại cho kết quả tốt nhất.

Kết quả tệ nhất trong tất cả các phương pháp. Điều này nhấn mạnh thực tế rằng ngay cả khi Dữ liệu thể hiện mối quan hệ phi tuyến tính phức tạp, một phương pháp phi tham số.

Ví dụ như thuật toán KNN vẫn có thể cho kết quả kém nếu mức độ làm mịn không được đảm bảo.

Đã chọn đúng.

Kịch bản 6: Các quan sát được tạo ra từ phân phối chuẩn.

với ma trận hiệp phương sai đường chéo khác nhau cho mỗi lớp. Tuy nhiên,

Kích thước mẫu rất nhỏ: chỉ n = 6 trong mỗi lớp. Thuật toán Naive Bayes đã thực hiện...

Rất tốt, vì các giả định của nó được đáp ứng. LDA và hồi quy logistic

hoạt động kém hiệu quả vì ranh giới quyết định thực sự không tuyến tính, do

các ma trận hiệp phương sai không bằng nhau. QDA hoạt động kém hơn một chút so với phương pháp đơn giản.

Phương pháp Bayes, vì với kích thước mẫu rất nhỏ, phương pháp trước đó đã dẫn đến quá nhiều sai sót.

Có nhiều sự khác biệt trong việc ước tính mối tương quan giữa các biến dự báo bên trong

mỗi lớp. Hiệu suất của KNN cũng bị ảnh hưởng do số lượng mẫu rất nhỏ.

kích cỡ.

Sáu ví dụ này minh họa rằng không có phương pháp nào vượt trội hơn các phương pháp khác

trong mọi tình huống. Khi ranh giới quyết định thực sự là tuyến tính, thì

Các phương pháp LDA và hồi quy logistic thường sẽ cho kết quả tốt. Khi

Các ranh giới có tính phi tuyến vừa phải, QDA hoặc Naive Bayes có thể cung cấp

kết quả tốt hơn. Cuối cùng, đối với các ranh giới quyết định phức tạp hơn nhiều,

Phương pháp phi tham số như KNN có thể vượt trội hơn. Nhưng mức độ của

Độ trơn tru của phương pháp phi tham số cần được lựa chọn cẩn thận. Trong

Chương tiếp theo, chúng ta sẽ xem xét một số phương pháp để lựa chọn phương pháp chính xác.

mức độ mượt mà và, nói chung, để lựa chọn phương pháp tổng thể tốt nhất.

Cuối cùng, hãy nhớ lại từ Chương 3 rằng trong thiết lập hồi quy, chúng ta có thể xử lý

mối quan hệ phi tuyến tính giữa các biến dự báo và biến phản hồi.

bằng cách thực hiện hồi quy sử dụng các phép biến đổi của các biến dự báo. Một cách tương tự

có thể áp dụng phương pháp này trong bối cảnh phân loại. Ví dụ, chúng ta có thể

	Hệ số Sai số chuẩn Thống kê t Giá trị p
Intercept	73,60 5,13 14,34 0,00
workingday	1,27 1,78 0,71 0,48
temp	157,21 10,26 15,32 0,00
weathersit[cloudy/smody]	-12,89 1,96 -6,56 0,00
weathersit[light rain/snow]	-66,49 2,97 -22,43 0,00
weathersit[heavy rain/snow]	-109,75 76,67 -1,43 0,15

BẢNG 4.10. Kết quả của mô hình tuyến tính bình phương tối thiểu được sử dụng để dự đoán số lượng người đi xe đạp trong Dữ liệu Bikeshare . Các biến dự báo month và hr đã bị lược bỏ khỏi bảng này vì do hạn chế về không gian, và có thể thấy trong Hình 4.13. Đối với biến định tính Theo Weathersit, mức cơ bản tương ứng với bầu trời quang đãng.

Tạo ra một phiên bản linh hoạt hơn của hồi quy logistic bằng cách bao gồm X2, X3, và thậm chí cả X4 làm biến dự đoán. Điều này có thể cải thiện hoặc không cải thiện hiệu suất của hồi quy logistic, tùy thuộc vào việc liệu sự gia tăng phương sai do Tính linh hoạt được tăng thêm được bù đắp bằng việc giảm đáng kể độ lệch. Chúng tôi Có thể làm điều tương tự với LDA. Nếu chúng ta cộng tất cả các số hạng bậc hai có thể có và tích chéo với LDA, dạng của mô hình sẽ giống như sau:
Mô hình QDA, mặc dù các ước tính tham số sẽ khác nhau. Điều này Thiết bị này cho phép chúng ta di chuyển đến một vị trí trung gian giữa mô hình LDA và QDA.

4.6 Mô hình tuyến tính tổng quát

Trong Chương 3, chúng ta đã giả định rằng biến phản hồi Y là định lượng và đã khám phá việc sử dụng hồi quy tuyến tính bình phương tối thiểu để dự đoán Y. Cho đến nay trong Trong chương này, chúng ta giả định rằng Y là định tính. Tuy nhiên, chúng ta Đôi khi có thể gặp phải những tình huống mà Y không phải là định tính. cũng không mang tính định lượng, và do đó không phải là hồi quy tuyến tính từ Chương 3 cũng không phải là... Các phương pháp phân loại được đề cập trong chương này đều có thể áp dụng.

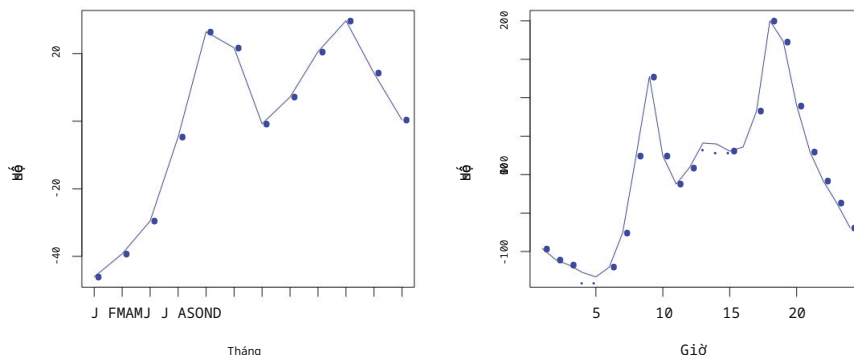
Để làm ví dụ cụ thể, chúng ta hãy xem xét tập dữ liệu Bikeshare . Phản hồi Đây là số lượng người sử dụng chương trình chia sẻ xe đạp hàng giờ tại Washington, DC. Giá trị phản hồi này không mang tính định tính cũng không mang tính định lượng: Thay vào đó, nó nhận các giá trị nguyên không âm, hay số lượng. Chúng ta sẽ xem xét việc dự đoán số lượng người đi xe đạp bằng cách sử dụng các biến phụ thuộc mnth (tháng trong năm), hr (giờ) của ngày, từ 0 đến 23), ngày làm việc (một biến chỉ báo bằng 1 nếu (không phải cuối tuần cũng không phải ngày lễ), nhiệt độ (nhiệt độ chuẩn hóa, (tính bằng độ C), và thời tiết (một biến định tính có thể nhận một trong bốn giá trị) Các giá trị có thể: trời quang; có sương mù hoặc nhiều mây; mưa nhẹ hoặc tuyết nhẹ; hoặc mưa lớn. (mưa hoặc tuyết rơi dày.)

Trong các phân tích tiếp theo, chúng ta sẽ coi mnth, hr và weathersit như sau: Các biến định tính.

4.6.1 Hồi quy tuyến tính trên dữ liệu chia sẻ xe đạp

Đầu tiên, chúng ta xem xét việc dự đoán số lượng người đi xe máy bằng phương pháp hồi quy tuyến tính. Kết quả được thể hiện trong Bảng 4.10.

Ví dụ, chúng ta thấy sự thay đổi thời tiết từ trời quang đãng sang nhiều mây. Kết quả là, trung bình, có ít hơn 12,89 người đi xe đạp mỗi giờ; tuy nhiên, nếu thời tiết Nếu tình hình tiếp tục chuyển biến thành mưa hoặc tuyết, thì lượng mưa sẽ giảm đi 53,60%. số người đi xe đạp mỗi giờ. Hình 4.13 hiển thị các hệ số liên quan đến tháng.



HÌNH 4.13. Một mô hình hồi quy tuyến tính bình phương tối thiểu đã được sử dụng để dự đoán số lượng người đi xe đạp trong tập dữ liệu **Bikeshare**. Bên trái: Các hệ số liên quan đến tháng của Năm. Lượng người sử dụng xe đạp cao nhất vào mùa xuân và mùa thu, và thấp nhất vào mùa đông. Bên phải: Các hệ số liên quan đến giờ trong ngày. Lượng người sử dụng xe đạp cao nhất vào lúc... Thời gian cao điểm đi làm và thấp nhất vào ban đêm.

và các hệ số liên quan đến giờ. Chúng ta thấy rằng tỷ lệ sử dụng xe đạp cao nhất ở Nhiệt độ cao nhất vào mùa xuân và mùa thu, và thấp nhất vào các tháng mùa đông. Hơn nữa, xe đạp Lượng người sử dụng cao nhất vào giờ cao điểm (9 giờ sáng và 6 giờ chiều), và thấp nhất vào ban đêm. Do đó, thoát nhìn, việc áp dụng mô hình hồi quy tuyến tính vào dữ liệu **Bikeshare** có vẻ khả thi. Phương pháp này dường như mang lại kết quả hợp lý và trực quan.

Nhưng khi xem xét kỹ hơn, một số vấn đề trở nên rõ ràng. Bởi vì

Ví dụ, 9,6% các giá trị được ước tính trong tập dữ liệu **Bikeshare** là âm:

Tức là, mô hình hồi quy tuyến tính dự đoán số lượng người dùng là âm.

Trong 9,6% số giờ trong tập dữ liệu, điều này đặt ra câu hỏi về khả năng của chúng ta.

để đưa ra những dự đoán có ý nghĩa trên dữ liệu, và điều này cũng làm dấy lên những lo ngại.

về độ chính xác của các ước tính hệ số, khoảng tin cậy và

Các kết quả đầu ra khác của mô hình hồi quy.

Hơn nữa, có lý do để nghi ngờ rằng khi giá trị kỳ vọng

Số lượng người đi xe máy ít, do đó sự khác biệt giữa những người đi xe máy cũng nên nhỏ.

Ví dụ, vào lúc 2 giờ sáng trong một trận bão tuyết lớn tháng 12, chúng ta dự đoán rằng

Rất ít người sẽ sử dụng xe đạp, và hơn nữa, lẽ ra phải có...

Sự biến động liên quan đến số lượng người dùng trong những điều kiện đó là rất nhỏ.

Điều này được chứng minh bằng số liệu: từ 1 giờ sáng đến 4 giờ sáng, trong tháng 12,

Tháng Giêng và tháng Hai, khi trời mưa, trung bình có 5,05 người dùng.

với độ lệch chuẩn là 3,73. Ngược lại, trong khoảng thời gian từ 7 giờ sáng đến 10 giờ sáng,

Vào tháng Tư, tháng Năm và tháng Sáu, khi trời quang đãng, có 243,59 người dùng.

trung bình, với độ lệch chuẩn là 131,7. Mối quan hệ giữa trung bình và phương sai

được hiển thị ở bảng bên trái của Hình 4.14. Đây là một vi phạm nghiêm trọng.

trong số các giả định của mô hình tuyến tính, trong đó nêu rằng $Y = X\beta + \epsilon$, $\epsilon_j = 1$

trong đó là số hạng sai số có giá trị trung bình bằng 0 và phương sai σ^2 là hằng số, và

không phải là một hàm của các biến độc lập. Do đó, hiện tượng phương sai không đồng nhất của

Dữ liệu đặt ra nghi vấn về tính phù hợp của mô hình hồi quy tuyến tính.

Cuối cùng, phần hồi của người đi xe máy là một giá trị nguyên. Nhưng theo mô hình tuyến tính,

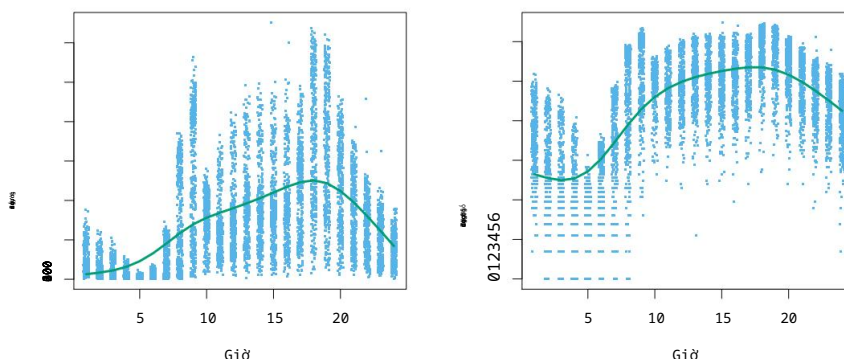
$Y = \beta_0 + X\beta + \epsilon$, trong đó là số hạng sai số có giá trị liên tục. Điều này

Điều này có nghĩa là trong mô hình tuyến tính, biến phản hồi Y nhất thiết phải có giá trị liên

tục (định lượng). Do đó, tính chất số nguyên của biến phản hồi "bikers" cho thấy mô hình hồi

quy tuyến tính không hoàn toàn phù hợp với dữ liệu này.

bộ.



HÌNH 4.14. Bên trái: Trên tập dữ liệu **Bikeshare**, số lượng người đi xe đạp được hiển thị trên trục y, và giờ trong ngày được hiển thị trên trục x. Độ lệch được áp dụng để dễ hình dung hơn. Phần lớn, vì số trung bình của số lượng người đi xe máy tăng lên, kéo theo sự biến động về số lượng người đi xe máy cũng tăng lên. Một đường cong spline làm mịn vùng phù hợp được hiển thị bằng màu xanh lá cây. Bên phải: Nhật ký số lượng người đi xe đạp hiện đang được hiển thị trên trục y.

Một số vấn đề phát sinh khi áp dụng mô hình hồi quy tuyến tính.

Vấn đề về dữ liệu **Bikeshare** có thể được khắc phục bằng cách chuyển đổi phản hồi; ví dụ: ví dụ, chúng ta có thể điều chỉnh mô hình

$$\log(Y) = \sum_{j=1}^p X_j \beta_j + \epsilon$$

Việc chuyển đổi câu trả lời giúp tránh khả năng đưa ra những dự đoán tiêu cực. và nó khắc phục được phần lớn hiện tượng phương sai không đồng nhất trong dữ liệu chưa được biến đổi. như được thể hiện ở bảng bên phải của Hình 4.14. Tuy nhiên, nó không hoàn toàn một giải pháp thỏa đáng, vì các dự đoán và suy luận được thực hiện dựa trên Nhật ký ghi lại phản hồi, chứ không phải chính phản hồi. Điều này dẫn đến những thách thức. trong diễn giải, ví dụ: “sự tăng một đơn vị của X_j có liên quan đến một sự gia tăng giá trị trung bình của logarit Y lên một lượng β_j ”. Hơn nữa, một Việc biến đổi logarit của phản hồi không thể áp dụng trong các trường hợp mà... Phản hồi có thể nhận giá trị bằng 0. Do đó, khi áp dụng mô hình tuyến tính cho Việc chuyển đổi phản hồi có thể là một cách tiếp cận phù hợp đối với một số trường hợp. Đối với các tập dữ liệu có giá trị đếm, nó thường không đáp ứng được kỳ vọng. Chúng ta sẽ thấy điều đó sau. Trong phần tiếp theo, mô hình hồi quy Poisson sẽ cung cấp một giải pháp toàn diện hơn nhiều. Một cách tiếp cận tự nhiên và trang nhã cho nhiệm vụ này.

4.6.2 Hồi quy Poisson trên dữ liệu chia sẻ xe đạp

Để khắc phục những hạn chế của hồi quy tuyến tính trong việc phân tích hệ thống chia sẻ xe đạp. Với tập dữ liệu này, chúng ta sẽ sử dụng một phương pháp thay thế, được gọi là phương pháp Poisson. Hồi quy Poisson. Trước khi nói về hồi quy Poisson, chúng ta phải giới thiệu phân phối Poisson trước.

Giả sử biến ngẫu nhiên Y nhận các giá trị nguyên không âm.

Tức là $Y \in \{0, 1, 2, \dots\}$. Nếu Y tuân theo phân phối Poisson thì

$$\Pr(Y = k) = \frac{e^{-\lambda} \lambda^k}{k!} \quad \text{với } k = 0, 1, 2, \dots \quad (4.35)$$



Poisson
hồi quy
Poisson
phân bố

170 4. Phân loại

Ở đây, $\lambda > 0$ là giá trị kỳ vọng của Y , tức là $E(Y)$. Hóa ra λ cũng bằng phương sai của Y , tức là $\lambda = E(Y) = \text{Var}(Y)$. Điều này có nghĩa là nếu Y tuân theo phân phối Poisson, do đó, giá trị trung bình của Y càng lớn thì giá trị trung bình càng lớn. phương sai của nó. (Trong (4.35), ký hiệu $k!$, được phát âm là “k giai thừa”, được định nghĩa vì $k! = k \times (k-1) \times (k-2) \times \dots \times 3 \times 2 \times 1$.)

Phân phối Poisson thường được sử dụng để mô hình hóa số lượng; đây là một lựa chọn tự nhiên vì một số lý do, bao gồm cả thực tế là số lượng, giống như Phân phối Poisson nhận các giá trị nguyên không âm. Để xem cách thức Trong thực tế, chúng ta có thể sử dụng phân phối Poisson, giả sử Y là số lượng người dùng chương trình chia sẻ xe đạp trong một giờ cụ thể. trong ngày, dưới một tập hợp các điều kiện thời tiết cụ thể và trong một tháng cụ thể của năm. Chúng ta có thể mô hình hóa Y như một phân phối Poisson với trung bình $E(Y) = \lambda = 5$. Điều này có nghĩa là xác suất không có người dùng nào trong thời gian xảy ra sự kiện vào giờ cụ thể này là $\Pr(Y = 0) = \frac{e^{-5} 5^0}{0!} = e^{-5} \approx 0,0067$ (trong đó $= 0) = 0! = 1$ theo quy ước). Xác suất có chính xác một người dùng là $\Pr(Y = 1) = \frac{e^{-5} 5^1}{1!} = 5e^{-5} \approx 0,034$, xác suất có hai người dùng là $\Pr(Y = 2) = \frac{e^{-5} 5^2}{2!} \approx 0,084$, và cứ thế tiếp tục.

Đĩ nhiên, trên thực tế, chúng ta kỳ vọng vào số lượng người dùng xe đạp trung bình. chương trình chia sẻ, $\lambda = E(Y)$, sẽ thay đổi theo giờ trong ngày, tháng trong năm, điều kiện thời tiết, vân vân. Vì vậy, thay vì... thay vì mô hình hóa số lượng người đi xe đạp, Y , như một phân phối Poisson với một Với giá trị trung bình cố định như $\lambda = 5$, chúng ta muốn cho phép giá trị trung bình thay đổi. hàm của các biến độc lập. Cụ thể, chúng ta xem xét mô hình sau: đối với giá trị trung bình $\lambda = E(Y)$, mà bây giờ chúng ta viết là $\lambda(X_1, \dots, X_p)$ để nhấn mạnh Điều đó có nghĩa là nó là một hàm của các biến độc lập $X_1, \dots, X_p$:

$$\log(\lambda(X_1, \dots, X_p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p \quad (4.36)$$

hoặc tương đương

$$\lambda(X_1, \dots, X_p) = e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}. \quad (4.37)$$

Ở đây, $\beta_0, \beta_1, \dots, \beta_p$ là các tham số cần ước tính. Cùng nhau, (4.35) và (4.36) xác định mô hình hồi quy Poisson. Lưu ý rằng trong (4.36), chúng ta lấy logarit của $\lambda(X_1, \dots, X_p)$ là tuyến tính theo $X_1, \dots, X_p$, thay vì có $\lambda(X_1, \dots, X_p)$ tự nó phải tuyến tính theo $X_1, \dots, X_p$; điều này đảm bảo rằng $\lambda(X_1, \dots, X_p)$ có giá trị không âm đối với tất cả các giá trị của biến độc lập.

Để ước tính các hệ số $\beta_0, \beta_1, \dots, \beta_p$, chúng ta sử dụng cùng một giá trị tối đa. phương pháp xác suất mà chúng tôi đã áp dụng cho hồi quy logistic trong Phần 4.3.2. Cụ thể, với n quan sát độc lập từ hồi quy Poisson mô hình, xác suất có dạng

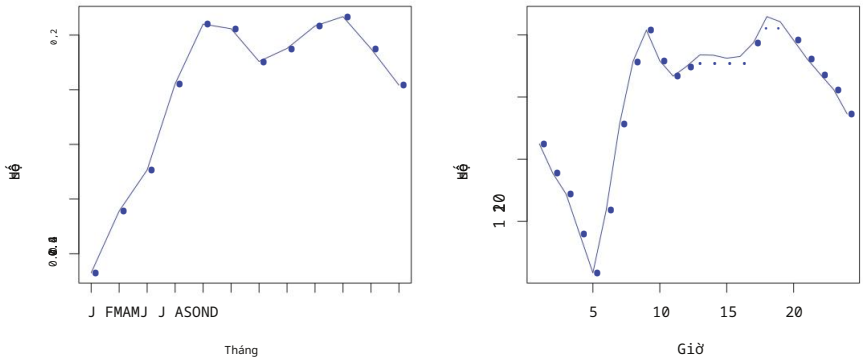
$$(\beta_0, \beta_1, \dots, \beta_p) = \frac{e^{-\sum_{i=1}^n \lambda(x_i)} \prod_{i=1}^n \lambda(x_i)^{y_i}}{\prod_{i=1}^n y_i!}, \quad (4.38)$$

trong đó $\lambda(x_i) = e^{\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}}$ do (4.37). Chúng tôi ước tính các hệ số làm tối đa hóa khả năng $(\beta_0, \beta_1, \dots, \beta_p)$, tức là làm cho Dữ liệu quan sát được càng chính xác càng tốt.

Chúng tôi hiện đang áp dụng mô hình hồi quy Poisson cho tập dữ liệu *Bikeshare*. Kết quả được trình bày trong Bảng 4.11 và Hình 4.15. Về mặt định tính, các kết quả Tương tự như kết quả từ hồi quy tuyến tính trong Phần 4.6.1. Chúng ta lại thấy Lượng người sử dụng xe đạp cao nhất vào mùa xuân và mùa thu, cũng như vào giờ cao điểm.

	Hệ số	Sai số chuẩn	Thống kê z	Giá trị p
Intercept	4.12	0.01	683.96	0.00
workingday temp	0,01	0,00	7,5	0,00
	0,79	0,01	68,43	0,00
weathersit[cloudy/smody]	-0.08	0.00	-34.53	0.00
weathersit[light rain/snow]	-0.58	0.00	-141.91	0.00
weathersit[heavy rain/snow]	-0,93	0,17	-5,55	0,00

BẢNG 4.11. Kết quả của mô hình hồi quy Poisson được sử dụng để dự đoán số lượng người đi xe đạp trong dữ liệu Bikeshare. Các biến dự báo month và hr đã bị lược bỏ khỏi bảng này vì do hạn chế về không gian, và có thể thấy trong Hình 4.15. Đối với biến định tính Theo Weathersit, đường cơ sở tương ứng với bầu trời quang đãng.



HÌNH 4.15. Mô hình hồi quy Poisson được sử dụng để dự đoán số lượng người đi xe đạp trong Bộ dữ liệu chia sẻ xe đạp. Bên trái: Các hệ số liên quan đến tháng trong năm. Lượng người sử dụng xe đạp cao nhất vào mùa xuân và mùa thu, và thấp nhất vào mùa đông. Bên phải: The các hệ số liên quan đến giờ trong ngày. Lượng người sử dụng xe đạp cao nhất vào giờ cao điểm. thời gian di chuyển và mức thấp nhất vào ban đêm.

và thấp nhất vào mùa đông và vào những giờ sáng sớm. Hơn nữa, Việc sử dụng xe đạp tăng lên khi nhiệt độ tăng và giảm xuống khi nhiệt độ giảm. Thời tiết xấu đi. Điều thú vị là hệ số liên quan đến ngày làm việc. có ý nghĩa thống kê theo mô hình hồi quy Poisson, nhưng không có ý nghĩa thống kê theo mô hình khác. Mô hình hồi quy tuyến tính.

Một số điểm khác biệt quan trọng giữa mô hình hồi quy Poisson và Mô hình hồi quy tuyến tính như sau:

- Giải thích: Để giải thích ý nghĩa của các hệ số trong hồi quy Poisson. mô hình, chúng ta phải chú ý kỹ đến (4.37), trong đó nêu rằng một Việc tăng X_j lên một đơn vị tương ứng với sự thay đổi trong $E(Y) = \lambda$ bởi một hệ số $\exp(\beta_j)$. Ví dụ, sự thay đổi thời tiết từ trời quang đãng Việc trời nhiều mây có liên quan đến sự thay đổi trong mức độ sử dụng xe đạp trung bình. hệ số $\exp(0,08) = 0,923$, tức là trung bình chỉ bằng 92,3% Mọi người sẽ sử dụng xe đạp nhiều hơn khi trời nhiều mây so với khi trời quang đãng. Nếu thời tiết xấu đi và bắt đầu mưa, thì mức trung bình sẽ... Việc sử dụng xe đạp sẽ tiếp tục thay đổi theo hệ số $\exp(0,5) = 0,607$, tức là Trung bình chỉ có 60,7% số người sử dụng xe đạp khi trời mưa. so với khi trời nhiều mây.

- Mối quan hệ giữa trung bình và phương sai: Như đã đề cập trước đó, theo mô hình Poisson, $\lambda = E(Y) = \text{Var}(Y)$. Do đó, bằng cách mô hình hóa việc sử dụng xe đạp bằng hồi quy Poisson, chúng ta ngầm giả định rằng mức sử dụng xe đạp trung bình trong một giờ nhất định bằng phương sai của việc sử dụng xe đạp trong giờ đó. Ngược lại, theo mô hình hồi quy tuyến tính, phương sai của việc sử dụng xe đạp luôn có giá trị không đổi. Hãy nhớ lại từ Hình 4.14 rằng trong dữ liệu **Bikeshare**, khi điều kiện đi xe đạp thuận lợi, cả trung bình và phương sai trong việc sử dụng xe đạp đều cao hơn nhiều so với khi điều kiện không thuận lợi. Do đó, mô hình hồi quy Poisson có thể xử lý mối quan hệ giữa trung bình và phương sai được thấy trong dữ liệu **Bikeshare** theo cách mà mô hình hồi quy tuyến tính không thể.
- Giá trị phù hợp không âm: Không có dự đoán âm nào khi sử dụng mô hình hồi quy Poisson. Điều này là do chính mô hình Poisson chỉ cho phép các giá trị không âm; xem (4.35). Ngược lại, khi chúng tôi áp dụng mô hình hồi quy tuyến tính cho tập dữ liệu **Bikeshare**, gần 10% dự đoán là âm.

sự phân tán quá mức

4.6.3 Mô hình tuyến tính tổng quát trong phạm vi rộng hơn

Chúng ta đã thảo luận về ba loại mô hình hồi quy: tuyến tính, logistic và Poisson. Các phương pháp này có một số đặc điểm chung:



- Mỗi phương pháp đều sử dụng các biến dự báo $X_1, \dots, X_p$ để dự đoán biến phản hồi Y . Chúng ta giả định rằng, với điều kiện $X_1, \dots, X_p$, Y thuộc về một họ phân phối nhất định. Đối với hồi quy tuyến tính, chúng ta thường giả định rằng Y tuân theo phân phối Gaussian hoặc phân phối chuẩn. Đối với hồi quy logistic, chúng ta giả định rằng Y tuân theo phân phối Bernoulli. Cuối cùng, đối với hồi quy Poisson, chúng ta giả định rằng Y tuân theo phân phối Poisson.
- Mỗi phương pháp đều mô hình hóa giá trị trung bình của Y như một hàm số của các biến dự báo. Trong hồi quy tuyến tính, giá trị trung bình của Y có dạng

$$E(Y | X_1, \dots, X_p) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p, \quad (4.39)$$

Tức là nó là một hàm tuyến tính của các biến dự báo. Đối với hồi quy logistic, giá trị trung bình thay vào đó có dạng

$$E(Y | X_1, \dots, X_p) = \Pr(Y = 1 | X_1, \dots, X_p) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}, \quad (4.40)$$

trong khi đó, đối với hồi quy Poisson, nó có dạng

$$E(Y | X_1, \dots, X_p) = \lambda(X_1, \dots, X_p) = e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}. \quad (4.41)$$

Các phương trình (4.39)-(4.41) có thể được biểu diễn bằng cách sử dụng một hàm liên kết, η , mà chức năng liên kết

Trên thực tế, phương sai trong dữ liệu **Bikeshare** dường như cao hơn nhiều so với giá trị trung bình, một tình trạng được gọi là phân tán quá mức. Điều này khiến các giá trị Z bị thổi phồng trong Bảng 4.11. Một phân tích cẩn thận hơn cần phải tính đến sự phân tán quá mức này để thu được các giá trị Z chính xác hơn, và có nhiều phương pháp để làm điều này. Nhưng chúng nằm ngoài phạm vi của cuốn sách này.

4.7 Thực hành: Hồi quy Logistic, LDA, QDA và KNN 173

áp dụng phép biến đổi cho $E(Y | X_1, \dots, X_p)$ sao cho giá trị trung bình đã biến đổi là một hàm tuyến tính của các biến dự báo. Tức là,

$$\eta(E(Y | X_1, \dots, X_p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p. \quad (4.42)$$

Các hàm liên kết cho hồi quy tuyến tính, logistic và Poisson lần lượt là $\eta(\mu) = \mu$, $\eta(\mu) = \log(\mu/(1-\mu))$ và $\eta(\mu) = \log(\mu)$.

Các phân phối Gaussian, Bernoulli và Poisson đều là thành viên của một lớp phân phối rộng hơn, được gọi là họ phân phối mũ. Các thành viên nổi tiếng khác của họ này là phân phối mũ, phân phối Gamma và phân phối nhị thức. Nói chung, chúng ta có thể thực hiện hồi quy bằng cách mô hình hóa biến phản hồi Y như đến từ một thành viên cụ thể của họ phân phối mũ, và sau đó biến đổi giá trị trung bình của biến phản hồi sao cho giá trị trung bình đã biến đổi là một hàm tuyến tính của các biến dự báo thông qua (4.42). Bất kỳ phương pháp hồi quy nào tuân theo công thức rất tổng quát này đều được gọi là mô hình tuyến tính tổng quát (GLM). Do đó, hồi quy tuyến tính, hồi quy logistic và hồi quy Poisson là ba ví dụ về GLM. Các ví dụ khác không được đề cập ở đây bao gồm hồi quy Gamma và hồi quy nhị thức âm.

họ hàm mũ
hàm mũ

Gamma
nhị thức
âm

mô hình tuyến
tính tổng quát

4.7 Thực hành: Hồi quy Logistic, LDA, QDA và KNN

4.7.1 Dữ liệu thị trường chứng khoán

Trong bài thực hành này, chúng ta sẽ xem xét dữ liệu *Smarket*, một phần của thư viện *ISLP*. Tập dữ liệu này bao gồm tỷ lệ lợi nhuận của chỉ số chứng khoán S&P 500 trong 1.250 ngày, từ đầu năm 2001 đến cuối năm 2005.

Đối với mỗi ngày, chúng tôi đã ghi lại tỷ lệ lợi nhuận phần trăm của năm ngày giao dịch trước đó, từ *Lag1* đến *Lag5*. Chúng tôi cũng đã ghi lại *Khối lượng* (số lượng cổ phiếu được giao dịch vào ngày hôm trước, tính bằng tỷ), *Ngày hôm nay* (tỷ lệ lợi nhuận phần trăm vào ngày được đề cập) và *Hướng* (thị trường tăng hay giảm vào ngày này).

Chúng ta bắt đầu bằng cách nhập các thư viện ở cấp cao nhất này; đây đều là các lệnh nhập.

Chúng ta đã thấy điều này trong các phòng thí nghiệm trước đây.

```
Trong [1]: import numpy as np
import pandas as pd from
matplotlib.pyplot import subplots import
statsmodels.api as sm from ISLP
import load_data from ISLP.models
import (ModelSpec as MS, summarize)
```

Chúng tôi cũng thu thập các vật tư nhập khẩu mới cần thiết cho phòng thí nghiệm này.

```
Trong [2]: from ISLP import confusion_table from
ISLP.models import contrast from
sklearn.discriminant_analysis import \
    (Phân tích phân biệt tuyến tính
    (LinearDiscriminantAnalysis) được gọi là LDA, Phân
    tích phân biệt bậc hai (QuadraticDiscriminantAnalysis) được
    gọi là QDA) từ sklearn.naive_bayes import GaussianNB từ sklearn.neighbors
import KNeighborsClassifier từ sklearn.preprocessing import StandardScaler
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

Giờ chúng ta đã sẵn sàng tải dữ liệu `Smarket`.

```
Trong [3]: Smarket = load_data('Smarket')
Smarket
```

Điều này cung cấp một danh sách dữ liệu rút gọn, mà chúng tôi không hiển thị ở đây. Chúng tôi có thể xem được tên các biến.

```
Trong [4]: Smarket.columns
```

```
Out[4]: Index(['Year', 'Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume',
              'Hôm nay', 'Hưởng đi'],
              dtype='object')
```

Chúng tôi tính toán ma trận tương quan bằng cách sử dụng phương thức `corr()` cho các khung dữ liệu, phương thức này tạo ra một ma trận chứa tất cả các tương quan từng cặp giữa các biến. (Chúng tôi lược bỏ kết quả ở đây.) Thư viện `pandas` mang tính định tính.

```
Trong [5]: Smarket.corr()
```

Như người ta vẫn thường thấy, mối tương quan giữa các biến lợi nhuận trễ và lợi nhuận ngày hôm nay gần như bằng không. Mối tương quan đáng kể duy nhất là mối quan hệ giữa `Năm` và `Khối lượng giao dịch`. Bằng cách vẽ biểu đồ dữ liệu, chúng ta thấy rằng `Khối lượng` giao dịch đang tăng lên theo thời gian. Nói cách khác, đó là số lượng cổ phiếu được giao dịch trung bình. Tăng hàng ngày từ năm 2001 đến năm 2005.

```
Trong [6]: Smarket.plot(y='Volume');
```

4.7.2 Hồi quy Logistic

Tiếp theo, chúng ta sẽ sử dụng mô hình hồi quy logistic để dự đoán `hướng đi`.

Sử dụng `Lag1` đến `Lag5` và `Volume`. Hàm `sm.GLM()` phù hợp với các mô hình tuyến tính tổng quát, một lớp mô hình bao gồm hồi quy logistic.

Ngoài ra, hàm `sm.Logit()` có thể trực tiếp khớp với mô hình hồi quy logistic. Cú pháp của `sm.GLM()` tương tự như `sm.OLS()`, ngoại trừ việc...

Chúng ta phải truyền vào đối số `family=sm.families.Binomial()` để

Hãy yêu cầu `statsmodels` chạy hồi quy logistic thay vì một loại hồi quy nào khác. mô hình tuyến tính tổng quát.

`sm.GLM()`

mô hình tuyến
tính tổng quát

```
Trong [7]: allvars = Smarket.columns.drop(['Hôm nay', 'Hưởng', 'Năm'])
          thiết kế = MS(tất cả các biến)
          X = design.fit_transform(Smarket)
          y = Smarket.Direction == 'Up'
          glm = sm.GLM(y,
                       X,
                       family=sm.families.Binomial())
          results = glm.fit()
          tóm tắt (kết quả)
```

4.7 Thực hành: Hồi quy Logistic, LDA, QDA và KNN 175

Ra ngoài[7]:

	hệ số	lỗi chuẩn	z	P> z
Hệ số chặn	-0.1260	Độ trễ 1	0,241 -0,523	0.601
-0.0731	Độ trễ 2	Độ trễ 3	Độ trễ	0.145
4 Độ	-0,0423	0,050 -0,845		0.398
trễ 5	0.0111	0,050	0.222	0.824
khối	0,0094	0,050	0.187	0.851
lượng	0,0103	0,050	0.208	0.835
	0.1354	0.158	0.855	0.392

Giá trị p nhỏ nhất ở đây liên quan đến Lag1. Hệ số âm Công cụ dự báo này cho thấy rằng nếu thị trường có lợi nhuận dương vào ngày hôm qua, Khi đó, khả năng nó tăng lên hôm nay sẽ thấp hơn. Tuy nhiên, ở giá trị 0,15, giá trị p Quy mô vẫn còn tương đối lớn, vì vậy chưa có bằng chứng rõ ràng về mối liên hệ thực sự. giữa Độ trễ 1 và Hướng.

Chúng tôi sử dụng thuộc tính 'params' của 'results' để truy cập trực tiếp các hệ số cho mô hình đã được hiệu chỉnh này.

Trong [8]: results.params

Out[8]:

intercept -0.126000
Độ trễ -0.073074
1 Độ -0.042301
trễ 2 0.011085
Độ trễ 0,009359
3 Độ 0.010313
trễ 4 Độ trễ 5 Âm lượng 0.135441
kiểu dữ liệu: float64

Tương tự, chúng ta có thể sử dụng thuộc tính pvalues để truy cập các giá trị p cho hệ số (không hiển thị).

Trong [9]: results.pvalues

Phương thức predict() của kết quả có thể được sử dụng để dự đoán xác suất. Thị trường sẽ tăng lên, dựa trên giá trị của các biến dự báo. Phương pháp này Trả về các dự đoán trên thang xác suất. Nếu không có tập dữ liệu nào được cung cấp cho Hàm predict() sau đó tính toán xác suất cho dữ liệu huấn luyện được sử dụng để phù hợp với mô hình hồi quy logistic. Tương tự như hồi quy tuyến tính, Trong hồi quy, người ta có thể truyền một đối số exog tùy chọn phù hợp với thiết kế. ma trận nếu cần. Ở đây chúng ta chỉ in ra mười xác suất đầu tiên.

Trong [10]: probs = results.predict()
các vấn đề[:10]

Out[10]:

array([0.5070841, 0.4814679, 0.4811388, 0.5152223, 0.5107812,
0.5069565, 0.4926509, 0.5092292, 0.5176135, 0.4888378])

Để đưa ra dự đoán về việc thị trường sẽ tăng hay giảm, cần phải dựa vào nhiều yếu tố. Vào một ngày cụ thể, chúng ta phải chuyển đổi những xác suất dự đoán này. thành các nhãn lớp, Lên hoặc Xuống. Hai lệnh sau tạo ra một vectơ các dự đoán về lớp dựa trên việc liệu xác suất dự đoán của một thị trường có đúng hay không. Mức tăng lớn hơn hoặc nhỏ hơn 0,5.

Trong [11]: labels = np.array(['Down']*1250)
nhãn[xác suất >0,5] = "Lên"

Hàm `confusion_table()` từ gói `ISLP` tóm tắt các dự đoán này, cho thấy có bao nhiêu quan sát được phân loại đúng hoặc sai. Hàm của chúng tôi, được điều chỉnh từ một hàm tương tự trong mô-đun `sklearn.metrics`, chuyển vị ma trận kết quả và bao gồm nhãn hàng và cột. Hàm `confusion_table()` nhận đối số đầu tiên là nhãn dự đoán và đối số thứ hai là nhãn thực.

```
Trong [12]: confusion_table(labels, Smarket.Direction)
```

```
Ra ngoài[12]:      Sự thật Xuống Lên
Dự đoán
Xuống 145 141
Lên 457 507
```

Các phần tử trên đường chéo chính của ma trận nhầm lẫn biểu thị các dự đoán chính xác, trong khi các phần tử ngoài đường chéo chính biểu thị các dự đoán không chính xác. Do đó, mô hình của chúng ta đã dự đoán chính xác rằng thị trường sẽ tăng vào 507 ngày và sẽ giảm vào 145 ngày, tổng cộng là $507 + 145 = 652$ dự đoán chính xác. Hàm `np.mean()` có thể được sử dụng để tính toán tỷ lệ số ngày mà dự đoán là chính xác. Trong trường hợp này, hồi quy logistic đã dự đoán chính xác sự biến động của thị trường trong 52,2% trường hợp.

```
Trong [13]: (507+145)/1250, np.mean(labels == Smarket.Direction)
```

```
Out[13]: (0.5216, 0.5216)
```

Thoạt nhìn, có vẻ như mô hình hồi quy logistic hoạt động tốt hơn một chút so với dự đoán ngẫu nhiên. Tuy nhiên, kết quả này gây hiểu nhầm vì chúng ta đã huấn luyện và kiểm tra mô hình trên cùng một tập hợp 1.250 quan sát. Nói cách khác, $100 \cdot 52,2 = 47,8\%$ là tỷ lệ lỗi huấn luyện. Như chúng ta đã thấy trước đây, tỷ lệ lỗi huấn luyện thường quá lạc quan – nó có xu hướng đánh giá thấp tỷ lệ lỗi kiểm thử. Để đánh giá tốt hơn độ chính xác của mô hình hồi quy logistic trong trường hợp này, chúng ta có thể sử dụng một phần dữ liệu để huấn luyện mô hình, sau đó kiểm tra xem nó dự đoán tốt như thế nào trên dữ liệu được giữ lại. Điều này sẽ cho ra tỷ lệ lỗi thực tế hơn, theo nghĩa là trên thực tế, chúng ta sẽ quan tâm đến hiệu suất của mô hình không phải trên dữ liệu mà chúng ta đã sử dụng để huấn luyện mô hình, mà là trên những ngày trong tương lai mà biến động của thị trường chưa được biết đến.

Để thực hiện chiến lược này, trước tiên chúng ta tạo một vectơ Boolean tương ứng với các quan sát từ năm 2001 đến năm 2004. Sau đó, chúng ta sử dụng vectơ này để tạo một tập dữ liệu kiểm chứng gồm các quan sát từ năm 2005.

```
Trong [14]: train = (Smarket.Year < 2005)
Smarket_train = Smarket.loc[train]
Smarket_test = Smarket.loc[ ~train]
Smarket_test.shape
```

```
Ra ngoài[14]: (252, 9)
```

Đối tượng `'train'` là một vectơ gồm 1.250 phần tử, tương ứng với các quan sát trong tập dữ liệu của chúng ta. Các phần tử của vectơ tương ứng với các quan sát xảy ra trước năm 2005 được đặt thành `True`, trong khi các phần tử tương ứng với các quan sát trong năm 2005 được đặt thành `False`. Do đó, `'train'` là một mảng boolean, vì các phần tử của nó là `True` và `False`. Mảng boolean có thể được sử dụng để lấy một tập hợp con các hàng hoặc cột của một khung dữ liệu bằng cách sử dụng...

Phương thức `loc` . Ví dụ, lệnh `Smarket.loc[train]` sẽ chọn ra một ma trận con của tập dữ liệu thị trường chứng khoán, chỉ tương ứng với các ngày trước năm 2005, vì đó là những ngày mà các phần tử của `train` có giá trị `True`. Ký hiệu `loc` có thể được sử dụng để phủ định tất cả các phần tử của một vectơ Boolean. Tức là, `train` là một vectơ tương tự như `train`, ngoại trừ các phần tử có giá trị `True` trong `train` được hoán đổi thành `False` trong `train`, và ngược lại. Do đó, `Smarket.loc[ ~train]` trả về một tập hợp con các hàng của khung dữ liệu thị trường chứng khoán chỉ chứa các quan sát mà `train` có giá trị `False`. Kết quả đầu ra ở trên cho thấy có 252 quan sát như vậy.

Chúng tôi tiến hành xây dựng mô hình hồi quy logistic chỉ sử dụng tập con các quan sát tương ứng với các ngày trước năm 2005. Sau đó, chúng tôi thu được xác suất dự đoán thị trường chứng khoán tăng cho mỗi ngày trong tập dữ liệu thử nghiệm của chúng tôi – tức là, cho các ngày trong năm 2005.

```
Trong [15]: X_train, X_test = X.loc[train], X.loc[ ~train] y_train, y_test =
           y.loc[train], y.loc[ ~train] glm_train = sm.GLM(y_train, X_train,
           family=sm.families.Binomial()) results =
           glm_train.fit() probs =
           results.predict(exog=X_test)
```

Xin lưu ý rằng chúng tôi đã huấn luyện và kiểm thử mô hình trên hai tập dữ liệu hoàn toàn riêng biệt: quá trình huấn luyện chỉ sử dụng dữ liệu trước năm 2005, và quá trình kiểm thử chỉ sử dụng dữ liệu trong năm 2005.

Cuối cùng, chúng ta so sánh các dự đoán cho năm 2005 với diễn biến thực tế của thị trường trong khoảng thời gian đó. Trước tiên, chúng ta sẽ lưu trữ nhãn kiểm tra và huấn luyện (nhớ rằng `y_test` là dữ liệu nhị phân).

```
Trong [16]: D = Smarket.Direction
           L_train, L_test = D.loc[train], D.loc[ ~train]
```

Bây giờ chúng ta đặt ngưỡng xác suất phù hợp ở mức 50% để tạo ra các nhãn dự đoán.

```
Trong [17]: labels = np.array(['Down']*252) labels[probs
           >0.5] = 'Up' confusion_table(labels,
           L_test)
```

```
Ra ngoài[17]:      Sự thật Xuống Lên
Dự đoán
Xuống      77  97
          34  44
```

Độ chính xác của bài kiểm tra khoảng 48% trong khi tỷ lệ sai sót khoảng 52%.

```
Trong [18]: np.mean(labels == L_test), np.mean(labels != L_test)
```

```
Out[18]: (0.4802, 0.5198)
```

Ký hiệu `!=` có nghĩa là không bằng, vì vậy lệnh cuối cùng tính toán tỷ lệ lỗi của tập dữ liệu thử nghiệm. Kết quả khá đáng thất vọng: tỷ lệ lỗi thử nghiệm là 52%, tệ hơn cả việc đoán ngẫu nhiên! Tất nhiên, kết quả này không quá ngạc nhiên, vì người ta thường không mong đợi có thể sử dụng lợi nhuận của những ngày trước để dự đoán hiệu suất thị trường trong tương lai.

(Xét cho cùng, nếu điều đó khả thi thì các tác giả cuốn sách này đã đi làm giàu thay vì viết sách giáo khoa thống kê.)

Chúng tôi nhớ lại rằng mô hình hồi quy logistic có giá trị p rất thấp đối với tất cả các biến dự đoán, và giá trị p nhỏ nhất, mặc dù không quá nhỏ, tương ứng với **Lag1**. Có lẽ bằng cách loại bỏ các biến dường như không hữu ích trong việc dự đoán **Hướng**, chúng ta có thể thu được một mô hình hiệu quả hơn. Xét cho cùng, việc sử dụng các biến dự đoán không có mối quan hệ với biến phản hồi có xu hướng làm suy giảm tỷ lệ lỗi kiểm định (vì các biến dự đoán như vậy làm tăng phương sai mà không làm giảm độ lệch tương ứng), và do đó việc loại bỏ các biến dự đoán như vậy có thể mang lại sự cải thiện. Dưới đây, chúng tôi hiệu chỉnh lại mô hình hồi quy logistic chỉ sử dụng **Lag1** và **Lag2**, dường như có sức mạnh dự đoán cao nhất trong mô hình hồi quy logistic ban đầu.

```
Trong [19]: model = MS(['Lag1', 'Lag2']).fit(Smarket)
X = model.transform(Smarket)
X_train, X_test = X.loc[train], X.loc[ train] glm_train =
sm.GLM(y_train, X_train,

family=sm.families.Binomial()) results =
glm_train.fit() probs =
results.predict(exog=X_test) labels =
np.array(['Down']*252) labels[probs >0.5] =
'Up' confusion_table(labels, L_test)
```

```
Ra ngoài[19]:      Sự thật Xuống Lên
Dự đoán
Xuống      35  35
          76 106
```

Hãy cùng đánh giá độ chính xác tổng thể cũng như độ chính xác trong những ngày mà hồi quy logistic dự đoán sự gia tăng.

```
Trong [20]: (35+106)/252, 106/(106+76)
```

```
Out[20]: (0.5595, 0.5824)
```

Hiện tại, kết quả có vẻ khả quan hơn một chút: 56% các biến động hàng ngày đã được dự đoán chính xác. Điều đáng chú ý là trong trường hợp này, một chiến lược đơn giản hơn nhiều là dự đoán thị trường sẽ tăng mỗi ngày cũng sẽ chính xác 56%! Do đó, xét về tỷ lệ lỗi tổng thể, phương pháp hồi quy logistic không tốt hơn phương pháp đơn giản. Tuy nhiên, ma trận nhầm lẫn cho thấy rằng vào những ngày mà hồi quy logistic dự đoán thị trường tăng, nó có tỷ lệ chính xác 58%. Điều này cho thấy một chiến lược giao dịch khả thi là mua vào vào những ngày mô hình dự đoán thị trường tăng và tránh giao dịch vào những ngày dự đoán giảm.

Dĩ nhiên, người ta cần phải điều tra kỹ lưỡng hơn để xem liệu sự cải thiện nhỏ này là có thật hay chỉ là do ngẫu nhiên.

Giả sử chúng ta muốn dự đoán lợi nhuận tương ứng với các giá trị cụ thể của **Lag1** và **Lag2**. Cụ thể, chúng ta muốn dự đoán **Direction** vào một ngày khi **Lag1** và **Lag2** lần lượt bằng 1,2 và 1,1, và vào một ngày khi chúng bằng 1,5 và -0,8. Chúng ta thực hiện điều này bằng cách sử dụng hàm `predict()`.

```
Trong [21]: newdata = pd.DataFrame({'Lag1':[1.2, 1.5], 'Lag2':[1.1, -0.8]});
```

```
newX = model.transform(newdata)
results.predict(newX)
```

```
Ra[21]: 0      0.4791
        1      0.4961
kiểu dữ liệu: float64
```

4.7.3 Phân tích phân biệt tuyến tính

Chúng tôi bắt đầu bằng cách thực hiện LDA trên dữ liệu `Smarket`, sử dụng hàm `LinearDiscriminantAnalysis()`, mà chúng tôi đã viết tắt là `LDA()`. Chúng tôi điều chỉnh mô hình chỉ sử dụng các quan sát trước năm 2005.

Tuyến tính
Phân biệt
Phân tích()

```
Trong [22]: lda = LDA(store_covariance=True)
```

Vì thuật toán ước lượng LDA tự động thêm hệ số chặn, chúng ta nên xóa cột tương ứng với hệ số chặn trong cả hai bảng `X_train` và `X_test`. Chúng ta cũng có thể sử dụng trực tiếp các nhãn thay vì các vectơ Boolean `y_train`.

```
Trong [23]: X_train, X_test = [M.drop(columns=['intercept']) for M in
                             [X_train, X_test]] lda.fit(X_train,
                                                         L_train)
```

```
Out[23]: LinearDiscriminantAnalysis(store_covariance=True)
```

Ở đây chúng ta đã sử dụng các biểu thức tạo danh sách được giới thiệu trong Mục 3.6.4. Nhìn vào dòng đầu tiên ở trên, ta thấy về bên phải là một danh sách có độ dài hai. Điều này là do đoạn mã `cho M trong [X_train, X_test]` lặp qua một danh sách có độ dài hai. Mặc dù ở đây chúng ta lặp qua một danh sách, phương thức `list comprehension` vẫn hoạt động khi lặp qua bất kỳ đối tượng có thể lặp nào. Sau đó, chúng ta áp dụng phương thức `drop()` cho từng phần tử trong vòng lặp, thu thập kết quả vào một danh sách. Về bên trái cho `Python` biết cần giải nén danh sách có độ dài hai này, gán các phần tử của nó cho các biến `X_train` và `X_test`. Tất nhiên, điều này sẽ ghi đè lên các giá trị trước đó của `X_train` và `X_test`.

.làm rồi()

Sau khi đã hiệu chỉnh mô hình, chúng ta có thể trích xuất giá trị trung bình trong hai lớp bằng thuộc tính `'means_'`. Đây là giá trị trung bình của mỗi biến dự báo trong mỗi lớp và được LDA sử dụng làm ước lượng của μ_k . Điều này cho thấy có xu hướng lợi nhuận của 2 ngày trước đó âm vào những ngày thị trường tăng và xu hướng lợi nhuận của những ngày trước đó dương vào những ngày thị trường giảm.

```
Trong [24]: lda.means_
```

```
Out[24]: array([[ 0.04,  0.03], [-0.04,
        -0.03]])
```

Các xác suất tiên nghiệm ước tính được lưu trữ trong thuộc tính `priors_`. Gói `sklearn` thường sử dụng dấu gạch dưới (`_`) này để biểu thị một đại lượng được ước tính khi sử dụng phương thức `fit()`. Chúng ta có thể chắc chắn mục nào tương ứng với nhãn nào bằng cách xem thuộc tính `classes_`.

```
Trong [25]: lda.classes_
```

```
Out[25]: array(['Down', 'Up'], dtype='<U4')
```


180 4. Phân loại

Kết quả đầu ra của LDA cho thấy $\pi^{\text{Down}} = 0,492$ và $\pi^{\text{Up}} = 0,508$.

```
Trong [26]: lda.priors_
```

```
Out[26]: array([0.492, 0.508])
```

Các vectơ phân biệt tuyến tính có thể được tìm thấy trong thuộc tính `scalings_` :

```
Trong [27]: lda.scalings_
```

```
Out[27]: array([[ -0.642], [ -0.513]])
```

Các giá trị này cung cấp tổ hợp tuyến tính của `Lag1` và `Lag2` được sử dụng để hình thành quy tắc quyết định LDA. Nói cách khác, đây là các hệ số nhân của các phần tử của $X = x$ trong (4.24). Nếu $0,64 \times \text{Lag1} - 0,51 \times \text{Lag2}$ lớn, thì bộ phân loại LDA sẽ dự đoán thị trường tăng, và nếu nó nhỏ, thì bộ phân loại LDA sẽ dự đoán thị trường giảm.

```
Trong [28]: lda_pred = lda.predict(X_test)
```

Như chúng ta đã thấy trong phần so sánh các phương pháp phân loại (Mục 4.5), Các dự đoán của LDA và hồi quy logistic gần như giống hệt nhau.

```
Trong [29]: confusion_table(lda_pred, L_test)
```

Ra ngoài[29]:

	Sự thật Xuống Lên
Dự đoán	
Xuống	35 35
Trên	76 106

Chúng ta cũng có thể ước tính xác suất của mỗi lớp cho mỗi điểm trong tập huấn luyện. Áp dụng ngưỡng 50% cho xác suất hậu nghiệm thuộc lớp một cho phép chúng ta tái tạo lại các dự đoán có trong `lda_pred`.

```
Trong [30]: lda_prob = lda.predict_proba(X_test)

np.all( np.where(lda_prob[:,1] >= 0.5, 'Up','Down') == lda_pred )
```

Ra ngoài[30]: Đúng

Ở trên, chúng ta đã sử dụng hàm `np.where()` để tạo một mảng với giá trị 'Up' cho các chỉ số mà cột thứ hai của `lda_prob` (xác suất hậu nghiệm ước tính của 'Up') lớn hơn 0,5. Đối với các bài toán có nhiều hơn hai lớp, nhãn được chọn là lớp có xác suất hậu nghiệm cao nhất:

`np.where()`

```
Trong [31]: np.all( [lda.classes[i] for i in np.argmax(lda_prob, 1)] ==
                lda_pred
            )
```

Ra ngoài[31]: Đúng

Nếu chúng ta muốn sử dụng ngưỡng xác suất hậu nghiệm khác 50% để đưa ra dự đoán, thì chúng ta hoàn toàn có thể làm được. Ví dụ, giả sử chúng ta chỉ muốn dự đoán thị trường giảm nếu chúng ta rất chắc chắn rằng...

Thị trường chắc chắn sẽ giảm vào ngày đó – chẳng hạn, nếu xác suất hậu nghiệm ít nhất là 90%. Chúng ta biết rằng cột đầu tiên của `lda_prob` tương ứng với nhãn `Down` sau khi đã kiểm tra thuộc tính `classes_`, do đó chúng ta sử dụng chỉ số cột 0 thay vì 1 như đã làm ở trên.

```
Trong [32]: np.sum(lda_prob[:,0] > 0.9)
```

```
Ra[32]: 0
```

Không có ngày nào trong năm 2005 đạt đến ngưỡng đó! Trên thực tế, xác suất hậu nghiệm lớn nhất về sự giảm trong cả năm 2005 là 52,02%.

Bộ phân loại LDA ở trên là bộ phân loại đầu tiên từ thư viện `sklearn`. Chúng ta sẽ sử dụng một số đối tượng khác từ thư viện này. Các đối tượng này tuân theo một cấu trúc chung giúp đơn giản hóa các tác vụ như kiểm định chéo, mà chúng ta sẽ thấy trong Chương 5. Cụ thể, các phương thức trước tiên tạo ra một bộ phân loại chung mà không tham chiếu đến bất kỳ dữ liệu nào. Sau đó, bộ phân loại này được huấn luyện trên dữ liệu bằng phương thức `fit()` và các dự đoán luôn được tạo ra bằng phương thức `predict()`.

Mô hình đầu tiên là khởi tạo bộ phân loại, sau đó huấn luyện nó, và cuối cùng là tạo ra các dự đoán là một lựa chọn thiết kế rõ ràng của `sklearn`. Tính đồng nhất này cho phép sao chép bộ phân loại một cách dễ dàng để có thể huấn luyện nó trên các tập dữ liệu khác nhau; ví dụ: các tập dữ liệu huấn luyện khác nhau phát sinh trong quá trình kiểm định chéo.

Mô hình tiêu chuẩn này cũng cho phép hình thành các quy trình làm việc một cách có thể dự đoán được.

4.7.4 Phân tích phân biệt bậc hai

Bây giờ chúng ta sẽ áp dụng mô hình QDA cho dữ liệu `Smarket`. QDA được triển khai thông qua hàm `QuadraticDiscriminantAnalysis()` trong gói `sklearn`, mà chúng ta viết tắt là `QDA()`. Cú pháp rất giống với `LDA()`.

bậc hai
Phân biệt
Phân tích()

```
Trong [33]: qda = QDA(store_covariance=True)
           qda.fit(X_train, L_train)
```

```
Out[33]: Phân tích phân biệt bậc hai (lưu trữ hiệp phương sai = Đúng)
```

Hàm `QDA()` sẽ tính toán lại các giá trị trung bình và các giá trị tiên nghiệm.

```
Trong [34]: qda.means_, qda.priors_
```

```
Out[34]: (array([[ 0.04279022,  0.03389409],
                  [-0.03954635, -0.03132544]]),
          array([0.49198397, 0.50801603]))
```

Bộ phân loại `QDA()` sẽ ước tính một ma trận hiệp phương sai cho mỗi lớp. Dưới đây là ma trận hiệp phương sai ước tính trong lớp đầu tiên:

```
Trong [35]: qda.covariance_[0]
```

```
Out[35]: array([[ 1.50662277, -0.03924806],
                  [-0.03924806, 1.53559498]])
```

Kết quả đầu ra chứa giá trị trung bình của nhóm. Tuy nhiên, nó không chứa các hệ số của các biến phân biệt tuyến tính, bởi vì bộ phân loại QDA sử dụng hàm bậc hai chứ không phải hàm tuyến tính đối với các biến dự đoán. Hàm `predict()` hoạt động hoàn toàn giống như đối với LDA.

```
Trong [36]: qda_pred = qda.predict(X_test)
           confusion_table(qda_pred, L_test)
```

```
Ra ngoài[36]:      Sự thật Xuống Lên
              Dự đoán
              Xuống      30 20
              Xuống      81 121
```

Điều thú vị là, các dự đoán của QDA chính xác gần 60% thời gian, ngay cả khi dữ liệu năm 2005 không được sử dụng để hiệu chỉnh mô hình.

```
Trong [37]: np.mean(qda_pred == L_test)
```

```
Out[37]: 0,599
```

Mức độ chính xác này khá ấn tượng đối với dữ liệu thị trường chứng khoán, vốn được biết đến là rất khó để mô hình hóa một cách chính xác. Điều này cho thấy dạng bậc hai mà QDA giả định có thể nắm bắt được mối quan hệ thực sự chính xác hơn so với dạng tuyến tính mà LDA và hồi quy logistic giả định. Tuy nhiên, chúng tôi khuyên bạn nên đánh giá hiệu suất của phương pháp này trên một tập dữ liệu thử nghiệm lớn hơn trước khi đặt cược rằng cách tiếp cận này sẽ liên tục vượt trội so với thị trường!

4.7.5 Thuật toán Naive Bayes

Tiếp theo, chúng ta sẽ áp dụng mô hình Naive Bayes cho dữ liệu `Smarket`. Cú pháp tương tự như của `LDA()` và `QDA()`. Theo mặc định, thuật toán `GaussianNB()` của bộ phân loại Naive Bayes mô hình hóa từng đặc trưng định lượng bằng cách sử dụng phân phối Gaussian. Tuy nhiên, phương pháp mật độ hạt nhân cũng có thể được sử dụng để ước tính các phân phối này. `GaussianNB()`

```
Trong [38]: NB = GaussianNB()
           NB.fit(X_train, L_train)
```

```
Out[38]: GaussianNB()
```

Các lớp được lưu trữ dưới dạng `classes_`.

```
Trong [39]: NB.classes_
```

```
Out[39]: array(['Down', 'Up'], dtype='<U4')
```

Xác suất tiên nghiệm của lớp được lưu trữ trong thuộc tính `class_prior_`.

```
Trong [40]: NB.class_prior_
```

```
Out[40]: array([0.49, 0.51])
```

Các tham số của các đặc trưng có thể được tìm thấy trong thuộc tính `theta_` và `var_`. Số hàng bằng số lớp, trong khi số cột bằng số đặc trưng. Chúng ta thấy bên dưới rằng giá trị trung bình của đặc trưng `Lag1` trong lớp `Down` là 0,043.

```
Trong [41]: NB.theta_
```

```
Out[41]: array([[ 0.043, 0.034], [-0.040,
              -0.031]])
```

Phương sai của nó là 1,503.

```
Trong [42]: NB.var_
```

```
Out[42]: array([[1.503, 1.532], [1.514,
                                1.487]])
```

Làm sao chúng ta biết tên của các thuộc tính này? Chúng ta sử dụng `NB?` (hoặc `?NB`). Chúng ta có thể dễ dàng kiểm chứng phép tính trung bình:

```
Trong [43]: X_train[L_train == 'Down'].mean()
```

```
Ra[43]: Độ trễ 1      0.042790
        Lag2        0.033894
        dtype: float64
```

Tương tự đối với phương sai:

```
Trong [44]: X_train[L_train == 'Down'].var(ddof=0)
```

```
Ra[44]: Độ trễ 1      1.503554
        Lag2        1.532467
        dtype: float64
```

Hàm `GaussianNB()` tính toán phương sai bằng công thức $1/n$. Vì `NB()` là một bộ phân loại trong thư viện `sklearn`, nên việc đưa ra dự đoán sử dụng cùng cú pháp như đối với `LDA()` và `QDA()` ở trên.

```
Trong [45]: nb_labels = NB.predict(X_test)
           confusion_table(nb_labels, L_test)
```

```
Ra ngoài[45]:      Sự thật Xuống Lên
              Dự đoán
              Xuống      29 20
              Xuống Lên  82 121
```

Thuật toán Naive Bayes hoạt động tốt trên tập dữ liệu này, với độ chính xác dự đoán trên 59%. Kết quả này hơi kém hơn so với QDA, nhưng tốt hơn nhiều so với LDA.

Đối với LDA, phương thức `predict_proba()` ước tính xác suất rằng mỗi quan sát thuộc về một lớp cụ thể.

```
Trong [46]: NB.predict_proba(X_test)[:5]
```

```
Out[46]: array([[0.4873, 0.5127], [0.4762,
                                0.5238], [0.4653,
                                0.5347], [0.4748,
                                0.5252], [0.4902,
                                0.5098]])
```

4.7.6 K-Hàng xóm gần nhất

Bây giờ chúng ta sẽ thực hiện thuật toán KNN bằng cách sử dụng hàm `KNeighborsClassifier()`. Điều này `KNeighbors` Bộ phân loại()

Có hai công thức để tính phương sai mẫu của n quan sát $i=1(x_i - \bar{x})^2$ trong đó $\bar{x}$ là trung bình mẫu. Trong hầu hết $\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$ và $x_1, \dots, x_n$: $\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ các trường hợp, sự phân biệt này không quan trọng.

Chức năng này hoạt động tương tự như các chức năng khớp mô hình khác mà chúng ta đã gặp cho đến nay.

Tương tự như LDA và QDA, chúng ta huấn luyện bộ phân loại bằng phương thức `'fit'`. Các dự đoán mới được tạo ra bằng phương thức `'predict'` của đối tượng được trả về bởi `'fit()'`.

```
Trong [47]: knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, L_train) knn1_pred =
knn1.predict(X_test) confusion_table(knn1_pred,
L_test)
```

Ra ngoài[47]: Sự thật Xuống Lên
Dự đoán
Xuống 43 58 68
83
hàng dự

Kết quả khi sử dụng $K = 1$ không được tốt lắm, vì chỉ có 50% số quan sát được dự đoán chính xác. Tất nhiên, cũng có thể là do $K = 1$ dẫn đến sự phù hợp quá linh hoạt với dữ liệu.

```
Trong [48]: (83+43)/252, np.mean(knn1_pred == L_test)
```

Out[48]: (0.5, 0.5)

Chúng tôi lặp lại phân tích bên dưới bằng cách sử dụng $K = 3$.

```
Trong [49]: knn3 = KNeighborsClassifier(n_neighbors=3)
knn3_pred = knn3.fit(X_train, L_train).predict(X_test) np.mean(knn3_pred == L_test)
```

Out[49]: 0.532

Kết quả đã được cải thiện đôi chút. Nhưng việc tăng K hơn nữa sẽ không mang lại cải thiện nào thêm. Có vẻ như với bộ dữ liệu này và cách chia tập huấn luyện/kiểm tra này, QDA cho kết quả tốt nhất trong số các phương pháp mà chúng ta đã xem xét cho đến nay.

Thuật toán KNN không hoạt động tốt trên tập dữ liệu `Smarket`, nhưng nó thường mang lại kết quả ấn tượng. Ví dụ, chúng ta sẽ áp dụng phương pháp KNN cho tập dữ liệu `Caravan`, một phần của thư viện `ISLP`. Tập dữ liệu này bao gồm 85 biến dự đoán đo lường các đặc điểm nhân khẩu học của 5.822 cá nhân.

Biến phản hồi là `"Mua hàng"`, cho biết liệu một cá nhân cụ thể có mua bảo hiểm xe caravan hay không. Trong tập dữ liệu này, chỉ có 6% người mua bảo hiểm xe caravan.

```
Trong [50]: Caravan = load_data('Caravan')
Mua hàng = Caravan.Mua hàng.Giá trị_số_lượng_mua
hàng()
```

Ra ngoài[50]: Không 5474
Đúng 348
Tên: Mua hàng, kiểu dữ liệu: int64

Phương thức `value_counts()` nhận vào một `pd.Series` hoặc `pd.DataFrame` và trả về một `pd.Series` chứa số lượng tương ứng cho mỗi phần tử duy nhất. Trong trường hợp này, `"Mua hàng"` chỉ có hai giá trị `"Có"` và `"Không"` và trả về số lượng của mỗi giá trị.

```
Trong [51]: 348 / 5822
```

```
Out[51]: 0.0598
```

Các tính năng của chúng tôi sẽ bao gồm tất cả các cột ngoại trừ cột **Mua hàng**.

```
Trong [52]: feature_df = Caravan.drop(columns=['Purchase'])
```

Vì thuật toán phân loại KNN dự đoán lớp của một quan sát thử nghiệm nhất định bằng cách xác định các quan sát gần nhất với nó, nên thang đo của các biến rất quan trọng. Bất kỳ biến nào có thang đo lớn sẽ có ảnh hưởng lớn hơn nhiều đến khoảng cách giữa các quan sát, và do đó ảnh hưởng đến thuật toán phân loại KNN, so với các biến có thang đo nhỏ. Ví dụ, hãy tưởng tượng một tập dữ liệu chứa hai biến, **lương** và **tuổi** (được đo bằng đô la và năm tương ứng). Đối với KNN, sự khác biệt 1.000 USD về lương là rất lớn so với sự khác biệt 50 năm về tuổi. Do đó, **lương** sẽ chi phối kết quả phân loại của KNN, và **tuổi** sẽ hầu như không có ảnh hưởng. Điều này trái ngược với trực giác của chúng ta rằng sự khác biệt 1.000 USD về lương là khá nhỏ so với sự khác biệt 50 năm về tuổi. Hơn nữa, tầm quan trọng của thang đo đối với thuật toán phân loại KNN dẫn đến một vấn đề khác: nếu chúng ta đo **lương** bằng yên Nhật, hoặc nếu chúng ta đo **tuổi** bằng phút, thì chúng ta sẽ nhận được kết quả phân loại khá khác so với kết quả nhận được nếu hai biến này được đo bằng đô la và năm.

Một cách hay để giải quyết vấn đề này là chuẩn hóa dữ liệu sao cho tất cả các biến đều có giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1. Khi đó, tất cả các biến sẽ nằm trên một thang đo tương đương. Điều này được thực hiện bằng cách sử dụng phép biến đổi **StandardScaler()**.

tiêu chuẩn hóa

```
Trong [53]: scaler = StandardScaler(with_mean=True,
                                   with_std=True,
                                   copy=True)
```

Tiêu chuẩn
Scaler()

Tham số **'with_mean'** cho biết liệu chúng ta có nên trừ đi giá trị trung bình hay không, trong khi **'with_std'** cho biết liệu chúng ta có nên chuẩn hóa các cột để có độ lệch chuẩn bằng 1 hay không. Cuối cùng, tham số **'copy=True'** cho biết rằng chúng ta sẽ luôn sao chép dữ liệu, thay vì cố gắng thực hiện các phép tính tại chỗ nếu có thể.

Phép biến đổi này có thể được điều chỉnh và áp dụng cho dữ liệu bất kỳ. Ở dòng đầu tiên bên dưới, các tham số để điều chỉnh tỷ lệ được tính toán và lưu trữ trong biến **scaler**, trong khi dòng thứ hai thực sự xây dựng tập hợp các đặc trưng đã được chuẩn hóa.

```
Trong [54]: scaler.fit(feature_df)
X_std = scaler.transform(feature_df)
```

Hiện tại, mỗi cột của **feature_std** bên dưới đều có độ lệch chuẩn bằng một và giá trị trung bình bằng không.

```
Trong [55]: feature_std = pd.DataFrame( X_std,
                                       columns=feature_df.columns);

feature_std.std()
```

```
Ra ngoài[55]: MOSTYPE      1.000086
              MAANTHUI     1.000086
```

MGEMOMV	1.000086
MGEMLEEF	1.000086
MOSHOOFD	1.000086
...	
AZEILPL	1.000086
APLEZIER	1.000086
AFIETS	1.000086
AINBOED	1.000086
VUC THAM	1.000086

Độ dài: 85, kiểu dữ liệu: float64

Lưu ý rằng độ lệch chuẩn ở đây không hoàn toàn bằng 1; điều này một lần nữa là do đối với một số quy trình sử dụng quy ước 1/n cho phương sai (trong trường hợp này một số sử dụng `scaler()`), trong khi những số khác sử dụng $1/(n-1)$ (phương pháp `std()`). Xem chú thích ở trang 183. Trong trường hợp này thì không quan trọng, miễn là tất cả các biến đều trên cùng một quy mô. `.std()`

Sử dụng hàm `train_test_split()`, chúng ta chia các quan sát thành một tập kiểm tra, chứa 1000 quan sát, và một tập huấn luyện, chứa các quan sát còn lại. `huấn_luyện_kiểm_thử_tách_ra()`

các quan sát còn lại. Tham số `random_state=0` đảm bảo rằng chúng ta nhận được

Việc phân chia vẫn giữ nguyên mỗi lần chúng ta chạy lại đoạn mã.

```
Trong [56]: (X_train,
           X_test,
           y_train,
           y_test) = train_test_split(feature_std,
                                     Mua,
                                     test_size=1000,
                                     random_state=0)
```

Hàm `train_test_split` cho thấy rằng các đối số không phải từ khóa có thể là danh sách mảng, dataframe pandas, v.v. tất cả đều có cùng độ dài (`hình_dạng[0]`) và Do đó, chúng có thể được lập chỉ mục. Trong trường hợp này, chúng là dataframe `feature_std` và biến phản hồi `Purchase`. Chúng tôi huấn luyện mô hình KNN trên dữ liệu huấn luyện. `có thể lập chỉ mục`

Sử dụng `K = 1`, và đánh giá hiệu suất của nó trên dữ liệu thử nghiệm.

```
Trong [57]: knn1 = KNeighborsClassifier(n_neighbors=1)
           knn1_pred = knn1.fit(X_train, y_train).predict(X_test)
           np.mean(y_test != knn1_pred), np.mean(y_test != "No")
```

Out[57]: (0.111, 0.067)

Tỷ lệ lỗi của thuật toán KNN trên 1.000 quan sát thử nghiệm là khoảng 11%. Ban đầu Thoạt nhìn, điều này có vẻ khá tốt. Tuy nhiên, vì chỉ hơn 6% Nhờ khách hàng mua bảo hiểm, chúng ta có thể giảm tỷ lệ lỗi xuống gần như bằng không. 6% bằng cách luôn dự đoán là **Không** bất kể giá trị của các biến dự đoán! Điều này được gọi là tỷ lệ không.

Giả sử việc bán bảo hiểm có một khoản chi phí không nhỏ. cho từng cá nhân cụ thể. Ví dụ, có thể một nhân viên bán hàng phải đến thăm từng người. khách hàng tiềm năng. Nếu công ty cố gắng bán bảo hiểm cho một người ngẫu nhiên. Nếu việc lựa chọn khách hàng không hiệu quả, thì tỷ lệ thành công chỉ là 6%, điều này có thể... Mức giá đó quá thấp so với chi phí phát sinh. Thay vào đó, công ty muốn Cố gắng bán bảo hiểm chỉ cho những khách hàng có khả năng mua bảo hiểm. Vì vậy, Tỷ lệ lỗi tổng thể không phải là điều đáng quan tâm. Thay vào đó, tỷ lệ phần trăm các cá nhân mà Việc dự đoán chính xác ai sẽ mua bảo hiểm là điều đáng quan tâm. `tỷ lệ bằng không`

```
Trong [58]: confusion_table(knn1_pred, y_test)
```

4.7 Thực hành: Hồi quy Logistic, LDA, QDA và KNN 187

```
Ra ngoài[58]:      Sự thật Không Có
                Dự đoán
                Số 880 58
                Vàng, 53      9
```

Kết quả cho thấy thuật toán KNN với K = 1 hoạt động hiệu quả hơn nhiều so với việc đoán ngẫu nhiên trong số những khách hàng được dự đoán sẽ mua bảo hiểm. Trong số 62 khách hàng như vậy, có 9 người, tương đương 14,5%, thực sự mua bảo hiểm. Tỷ lệ này gấp đôi so với tỷ lệ thu được từ việc đoán ngẫu nhiên.

```
Trong [59]: 9/(53+9)
```

Out[59]: 0.145

Thông số điều chỉnh

Số lượng láng giềng trong KNN được gọi là tham số điều chỉnh, hay còn gọi là siêu tham số. Chúng ta không biết trước giá trị nào cần sử dụng. Do đó, việc quan tâm đến hiệu suất của bộ phân loại trên dữ liệu thử nghiệm khi chúng ta thay đổi các tham số này là rất quan trọng. Điều này có thể được thực hiện bằng vòng lặp **for** , được mô tả trong Mục 2.3.8. Ở đây, chúng ta sử dụng vòng lặp **for** để xem xét độ chính xác của bộ phân loại trong nhóm được dự đoán sẽ mua bảo hiểm khi chúng ta thay đổi số lượng láng giềng từ 1 đến 5:

tham số
điều chỉnh

siêu
tham số

```
Trong [60]: đối với K trong phạm vi (1,6):
            knn = KNeighborsClassifier(n_neighbors=K) knn_pred =
            knn.fit(X_train, y_train).predict(X_test)
            C = confusion_table(knn_pred, y_test) templ = ('K={0:d}:
            # dự đoán sẽ thuê: {1:>2},' + # ai đã thuê {2:d}, độ chính xác {3:.1%}') pred
            = C.loc['Yes'].sum() did_rent = C.loc['Yes', 'Yes']
            print(templ.format(
                K,
                pred,
                did_rent,
                did_rent / pred))
```

K=1: Số người dự đoán sẽ thuê: 62, số người đã thuê: 9, độ chính xác 14,5% K=2: Số người dự đoán sẽ thuê: 6, số người đã thuê: 1, độ chính xác 16,7% K=3: Số người dự đoán sẽ thuê: 20, số người đã thuê: 3, độ chính xác 15,0% K=4: Số người dự đoán sẽ thuê: 3, số người đã thuê: 0, độ chính xác 0,0% K=5: Số người dự đoán sẽ thuê: 7, số người đã thuê: 1, độ chính xác 14,3%

Chúng ta thấy có một số sự khác biệt – các con số đối với K=4 rất khác so với phần còn lại.

So sánh với hồi quy logistic: Để so

sánh, chúng ta cũng có thể áp dụng mô hình hồi quy logistic cho dữ liệu. Điều này cũng có thể được thực hiện với **sklearn**, mặc dù theo mặc định nó sẽ phù hợp với một dạng hồi quy Ridge thực của hồi quy logistic, mà chúng ta sẽ giới thiệu trong Chương 6. Điều này có thể được sửa đổi bằng cách thiết lập đối số **C** bên dưới một cách thích hợp. Giá trị mặc định của nó là 1, nhưng bằng cách đặt nó thành một số rất lớn, thuật toán sẽ hội tụ về cùng một giải pháp như ước lượng hồi quy logistic thông thường (không được điều chỉnh) đã thảo luận ở trên.

Khác với gói `statsmodels`, `sklearn` tập trung ít hơn vào suy luận và nhiều hơn vào phân loại. Do đó, các phương thức **tóm tắt** được thấy trong `statsmodels` và phiên bản đơn giản hóa của chúng ta được thấy với `summarize` thường không khả dụng cho các bộ phân loại trong `sklearn`.

```
Trong [61]: logit = LogisticRegression(C=1e10, solver='liblinear')
logit.fit(X_train, y_train) logit_pred =
logit.predict_proba(X_test) logit_labels = np.where(logit_pred[:,1]
> 5, 'Yes', 'No') confusion_table(logit_labels, y_test)
```

```
Ra ngoài[61]:      Sự thật Không Có
Dự đoán
      Số 933 67
Đúng      0      0
```

Chúng tôi đã sử dụng đối số `solver='liblinear'` ở trên để tránh cảnh báo với trình giải mặc định, cảnh báo này sẽ cho biết rằng thuật toán không tương thích.
là đường.

Nếu chúng ta sử dụng 0,5 làm ngưỡng xác suất dự đoán cho bộ phân loại, thì chúng ta sẽ gặp vấn đề: không có quan sát nào trong số các quan sát thử nghiệm được dự đoán là sẽ mua bảo hiểm. Tuy nhiên, chúng ta không bắt buộc phải sử dụng ngưỡng 0,5. Nếu thay vào đó, chúng ta dự đoán việc mua hàng bất cứ khi nào xác suất mua hàng được dự đoán vượt quá 0,25, chúng ta sẽ nhận được kết quả tốt hơn nhiều: chúng ta dự đoán rằng 29 người sẽ mua bảo hiểm, và chúng ta đúng với khoảng 31% trong số những người này. Điều này tốt hơn gần năm lần so với việc đoán ngẫu nhiên!

```
Trong [62]: logit_labels = np.where(logit_pred[:,1]>0.25, 'Có', 'Không') confusion_table(logit_labels,
y_test)
```

```
Ra ngoài[62]:      Sự thật Không Có
Dự đoán
      Số 913 58
Vâng, 20      9
```

```
Trong [63]: 9/(20+9)
```

```
Out[63]: 0.310
```

4.7.7 Hồi quy tuyến tính và hồi quy Poisson trên dữ liệu chia sẻ xe đạp

Ở đây, chúng tôi áp dụng các mô hình hồi quy tuyến tính và Poisson cho dữ liệu `Bikeshare`, như đã mô tả trong Phần 4.6. Biến phản hồi "**người đi xe đạp**" đo lường số lượng xe đạp được thuê mỗi giờ tại Washington, DC trong giai đoạn 2010-2012.

```
Trong [64]: Bike = load_data('Bikeshare')
```

Hãy cùng xem qua kích thước và tên của các biến trong dataframe này.

```
Trong [65]: Bike.shape, Bike.columns
```

4.7 Thực hành: Hồi quy Logistic, LDA, QDA và KNN 189

```
Ra[65]: ((8645, 15),
        Chỉ mục(['mùa', 'tháng', 'ngày', 'giờ', 'ngày lễ', 'ngày trong tuần',
                  'ngày làm việc', 'thời tiết', 'nhiệt độ', 'nhiệt độ', 'tiếng ồn',
                  'tốc độ gió', 'thông thường', 'đã đăng ký', 'người đi xe máy'],
        dtype='object'))
```

Hồi quy tuyến tính

Chúng ta bắt đầu bằng cách áp dụng mô hình hồi quy tuyến tính vào dữ liệu.

```
Trong [66]: X = MS(['mnth',
                  'giờ',
                  'ngày làm việc',
                  'nhiệt độ',
                  'weathersit']).fit_transform(Bike)
Y = Xe đạp['người đi xe đạp']
M_lm = sm.OLS(Y, X).fit()
tóm tắt(M_lm)
```

```
Ra ngoài[66]:          hệ số lỗi chuẩn t P>|t|
hệ số chặn -68,6317 5,307 -12,932 0,000
tháng[Tháng 2] 6,8452 4,287 1,597 0,110
tháng tháng[tháng 3] 16,5514          4.301 3.848 0.000
tháng tháng[tháng 4] 41,4249          4.972 8.331 0.000
tháng tháng[tháng 5] 72,5571          5.641 12.862 0.000
tháng tháng[tháng 6] 67,8187          6.544 10,364 0,000
tháng tháng[tháng 7] 45,3245          7.081 6.401 0.000
tháng tháng[tháng 8] 53,2430          6.640 8.019 0.000
tháng tháng[tháng 9] 66,6783          5,925 11,254 0,000
tháng tháng[tháng 10] 75,8343          4.950 15.319 0.000
tháng[Tháng          60.3100          4.610 13.083 0.000
11] tháng[Tháng          46,4577          4.271 10.878 0.000
12]          -14,5793          5,699 -2,558 0,011
giờ[1]          -21,5791          5,733 -3,764 0,000
giờ[2] giờ[3]          -31.1408          5.778 -5.389 0.000
.....          .....          .....          .....
```

Có 24 cấp độ trong `bảng hr` và tổng cộng 40 hàng, vì vậy chúng tôi đã cắt bớt phần dữ liệu. Tóm tắt. Trong `M_lm`, các cấp độ đầu tiên `hr[0]` và `mnth[Jan]` được coi là các giá trị cơ sở, và do đó không có ước tính hệ số nào được cung cấp cho chúng: một cách ngầm định, ước tính hệ số của chúng bằng không, và tất cả các mức độ khác đều được đo lường. so với các mức cơ sở này. Ví dụ, hệ số tháng Hai là 6,845 cho thấy rằng, khi giữ tất cả các biến số khác không đổi, trung bình có khoảng 6,845 người đi xe đạp trong tháng Hai nhiều hơn tháng Giêng 7 người. Tương tự, tổng số người đi xe đạp là khoảng 16,5. Số lượng người đi xe đạp vào tháng 3 nhiều hơn tháng 1.

Kết quả được trình bày trong Phần 4.6.1 sử dụng cách mã hóa hơi khác một chút. các biến `hr` và `month`, như sau:

```
Trong [67]: hr_encode = contrast('hr', 'sum')
          mnth_encode = độ tương phản('mnth', 'tổng')
```

Lắp ráp lại lần nữa:

```
Trong [68]: X2 = MS([mnth_encode,
                  hr_encode,
                  'ngày làm việc',
                  'nhiệt độ',
```

```
'weathersit')).fit_transform(Bike)
M2_lm = sm.OLS(Y, X2).fit()
S2 = summarize(M2_lm)
S2
```

Ra ngoài[68]:

	hệ số lỗi chuẩn	t	P> t
chặn mnth[Jan]	73,5974	5.132	14.340 0.000
mnth[Feb]	-46,0871	4.085	-11.281 0.000
-39,2419 mnth[March]	-29,5357	3,539	-11,088 0,000
mnth[April] -4,6622 mnth[May]		3.155	-9.361 0.000
26,4700 mnth[Jun] 21,7317		2,741	-1,701 0,089
mnth[July] -0,7626 mnth[Aug]		2.851	9.285 0.000
7,1560 mnth[Sept] 20,5912		3,465	6,272 0,000
tháng[Oct] tháng[Nov] giờ[0]		3,908	-0,195 0,845
giờ[1] giờ[2]		3,535	2,024 0,043
		3.046	6.761 0.000
	29.7472	2.700	11.019 0.000
	14.2229	2.860	4.972 0.000
	-96.1420	3,955	-24,307 0,000
	-110.7213	3,966	-27,916 0,000
	-117.7212	4.016	-29.310 0.000
.....			

Sự khác biệt giữa hai cách mã hóa là gì? Trong `M2_lm`, hệ số ước tính được báo cáo cho tất cả các cấp độ trừ cấp độ `23` của `hr` và cấp độ `Dec` của `month`. Điều quan trọng là, Trong `M2_lm`, ước tính hệ số (không được báo cáo) cho cấp độ cuối cùng của `mnth` là không phải là số 0: thay vào đó, nó bằng giá trị âm của tổng các ước lượng hệ số cho tất cả các cấp độ khác. Tương tự, trong `M2_lm`, ước lượng hệ số Đối với cấp độ cuối cùng của `hr`, đó là giá trị âm của tổng các ước lượng hệ số. cho tất cả các cấp độ khác. Điều này có nghĩa là các hệ số của `hr` và `mnth` trong `M2_lm` sẽ luôn có tổng bằng 0 và có thể được hiểu là hiệu số. so với mức trung bình. Ví dụ, hệ số của tháng Giêng là `-46,087`. Điều này cho thấy rằng, khi giữ nguyên tất cả các biến số khác, thông thường sẽ có 46 Số lượng người đi xe đạp trong tháng Giêng ít hơn so với mức trung bình cả năm. Điều quan trọng cần nhận ra là việc lựa chọn ngôn ngữ lập trình thực sự không quan trọng. với điều kiện là chúng ta diễn giải chính xác kết quả đầu ra của mô hình dựa trên mã lập trình. được sử dụng. Ví dụ, chúng ta thấy rằng các dự đoán từ mô hình tuyến tính là Giống nhau bất kể phương pháp lập trình nào:

```
Trong [69]: np.sum((M_lm.fittedvalues - M2_lm.fittedvalues)**2)
```

Out[69]: 1.53e-20

Tổng bình phương các hiệu số bằng không. Chúng ta cũng có thể thấy điều này bằng cách sử dụng Hàm `np.allclose()` :

```
Trong [70]: np.allclose(M_lm.fittedvalues , M2_lm.fittedvalues) np.allclose()
```

Ra ngoài[70]: Đúng

Để tái tạo phần bên trái của Hình 4.13 , trước tiên chúng ta phải thu được Các ước tính hệ số liên quan đến `tháng`. Các hệ số cho tháng Giêng Thông tin này có thể được lấy trực tiếp từ đối tượng `M2_lm` cho đến tháng 11 . Hệ số cho tháng 12 phải được tính toán rõ ràng dưới dạng tổng âm. của tất cả các tháng khác. Đầu tiên, chúng ta trích xuất tất cả các hệ số cho tháng từ các hệ số của `M2_lm`.

4.7 Thực hành: Hồi quy Logistic, LDA, QDA và KNN 191

```
Trong [71]: coef_month = S2[S2.index.str.contains('mnth')]['coef']
           hệ số tháng
```

```
Ra[71]: mnth[Jan]                -46,0871
        mnth[Feb] -39.2419
        tháng [tháng 3] -29,5357
        mnth[April]            -4,6622
        mnth[May]              26.4700
        mnth[Tháng 6]          21.7317
        mnth[Tháng 7]          -0.7626
        mnth[Aug]              7.1560
        mnth[Sept]             20.5912
        mnth[Oct]              29.7472
        mnth[Nov]              14.2229
        Tên: coef, kiểu dữ liệu: float64
```

Tiếp theo, chúng ta thêm tháng 12 bằng cách lấy giá trị âm của tổng tất cả các tháng khác.

```
Trong [72]: months = Bike['mnth'].dtype.categories
           coef_month = pd.concat([
                                   hệ số_tháng,
                                   pd.Series([-coef_month.sum()],
                                             chỉ mục=['tháng[Tháng 12]'
                                             ])
                                   ])
           hệ số tháng
```

```
Ra[72]: mnth[Jan]                -46,0871
        mnth[Feb]                -39,2419
        mnth[March] -29.5357
        tháng [tháng Tư] -4,6622
        mnth[Tháng 5]           26.4700
        mnth[Tháng 6]           21.7317
        mnth[Tháng 7]           -0.7626
        mnth[Tháng 8]           7.1560
        mnth[Tháng 9]           20.5912
        mnth[Tháng 10]          29.7472
        10] mnth[Tháng 11]       14.2229
        11] mnth[Tháng 12]       0.3705
        Tên: coef, kiểu dữ liệu: float64
```

Cuối cùng, để sơ đồ gọn gàng hơn, chúng ta sẽ chỉ sử dụng chữ cái đầu tiên của mỗi tháng.

Đây là mục thứ 6 của mỗi nhãn trong chỉ mục.

```
Trong [73]: fig_month, ax_month = subplots(figsize=(8,8))
           x_month = np.arange(coef_month.shape[0])
           ax_month.plot(x_month, coef_month, marker='o', ms=10)
           ax_month.set_xticks(x_month)
           ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
                                     =20)
           ax_month.set_xlabel('Tháng', fontsize=20)
           ax_month.set_ylabel('Hệ số', fontsize=20);
```

Việc tái tạo đồ thị bên phải trong Hình 4.13 cũng tuân theo quy trình tương tự.

```
Trong [74]: coef_hr = S2[S2.index.str.contains('hr')]['coef']
           coef_hr = coef_hr.reindex(['hr[{}]' .format(h) for h in range(23)])
           coef_hr = pd.concat([coef_hr,
```

```
pd.Series([-coef_hr.sum()], index=['hr[23]']) ])
```

Bây giờ chúng ta lập biểu đồ theo giờ.

```
Trong [75]: fig_hr, ax_hr = subplots(figsize=(8,8))
x_hr = np.arange(coef_hr.shape[0]) ax_hr.plot(x_hr,
coef_hr, marker='o', ms=10) ax_hr.set_xticks(x_hr[::2])
ax_hr.set_xticklabels(range(24)[::2],
fontsize=20) ax_hr.set_xlabel('Giờ', fontsize=20) ax_hr.set_ylabel('Hệ
số', fontsize=20);
```

Hồi quy Poisson

Giờ đây, chúng ta sẽ sử dụng mô hình hồi quy Poisson cho dữ liệu *Bikeshare*. Hầu như không có thay đổi nào, ngoại trừ việc chúng ta sử dụng hàm `sm.GLM()` với họ Poisson được chỉ định:

```
Trong [76]: M_pois = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()
```

Chúng ta có thể vẽ đồ thị các hệ số liên quan đến tháng và giờ để Sao chép Hình 4.15. Trước tiên, chúng ta hoàn thành các hệ số này như trước.

```
Trong [77]: S_pois = summarize(M_pois)
coef_month = S_pois[S_pois.index.str.contains('mnth')]['coef'] coef_month =
pd.concat([coef_month,
coef_hr =
S_pois[S_pois.index.str.contains('hr')]
[ 'coef'] coef_hr = pd.concat([ coef_hr, pd.Series([-coef_hr.sum()],
index=['hr[23]'])])
```

Cốt truyện vẫn như cũ.

```
Trong [78]: fig_pois, (ax_month, ax_hr) = subplots(1, 2, figsize=(16,8))
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
=20)
ax_month.set_xlabel('Tháng', fontsize=20)
ax_month.set_ylabel('Hệ số', fontsize=20) ax_hr.plot(x_hr, coef_hr,
marker='o', ms=10) ax_hr.set_xticklabels(range(24)[::2],
fontsize=20) ax_hr.set_xlabel('Giờ', fontsize=20) ax_hr.set_ylabel('Hệ
số', fontsize=20);
```

Chúng tôi so sánh các giá trị dự đoán của hai mô hình. Các giá trị dự đoán được lưu trữ trong thuộc tính `fittedvalues` do phương thức `fit()` trả về cho cả hồi quy tuyến tính và phân phối Poisson. Các biến dự đoán tuyến tính được lưu trữ dưới dạng thuộc tính `lin_pred`.

```
Trong [79]: fig, ax = subplots(figsize=(8, 8))
ax.scatter(M2_lm.fittedvalues
M_pois.fittedvalues,
s=20)
ax.set_xlabel('Linear Regression Fit', fontsize=20)
```

```
ax.set_ylabel(' Kết quả hồi quy Poisson', fontsize=20)
ax.axline([0,0], c='đen', độ rộng đường kẻ=3,
          linestyle='--', slope=1);
```

Các dự đoán từ mô hình hồi quy Poisson có mối tương quan với Những giá trị từ mô hình tuyến tính; tuy nhiên, các giá trị trước đó là không âm. Như một Kết quả là, các dự đoán hồi quy Poisson có xu hướng lớn hơn so với các dự đoán từ các phương pháp khác. Mô hình tuyến tính áp dụng cho cả mức độ sử dụng dịch vụ rất thấp hoặc rất cao.

Trong phần này, chúng ta sẽ sử dụng hàm `sm.GLM()` để hiệu chỉnh mô hình hồi quy Poisson với đối số `family=sm.families.Poisson()`. Trước đó trong bài thực hành này, chúng ta sẽ thực hiện điều này. Chúng tôi đã sử dụng hàm `sm.GLM()` với tham số `family=sm.families.Binomial()` để thực hiện hồi quy logistic. Có thể sử dụng các lựa chọn khác cho tham số `family` để phù hợp với các loại GLM khác. Ví dụ, `family=sm.families.Gamma()` phù hợp Mô hình hồi quy Gamma.

4.8 Bài tập

Khái niệm

- Sử dụng một chút đại số, hãy chứng minh rằng (4.2) tương đương với (4.3). Trong Nói cách khác, biểu diễn hàm logistic và biểu diễn logit cho mô hình hồi quy logistic là tương đương nhau.
- Vấn đề đã nêu rõ việc phân loại một quan sát vào lớp. trong đó (4.17) lớn nhất tương đương với việc phân loại một quan sát đến lớp mà (4.18) là lớn nhất. Chứng minh rằng điều này là đúng. Trong Nói cách khác, với giả định rằng các quan sát ở vị trí thứ k các lớp được rút ra từ phân phối $N(\mu_k, \sigma^2)$, bộ phân loại Bayes gán một quan sát vào lớp mà hàm phân biệt áp dụng. được tối đa hóa.
- Vấn đề này liên quan đến mô hình QDA, trong đó các quan sát Các giá trị trong mỗi lớp được lấy từ phân phối chuẩn với vectơ trung bình đặc trưng cho lớp đó và ma trận hiệp phương sai đặc trưng cho lớp đó. Chúng ta xem xét trường hợp đơn giản khi $p = 1$; tức là chỉ có một đặc trưng. Giả sử chúng ta có K lớp, và nếu một quan sát thuộc về đến lớp thứ k thì X xuất phát từ một phân phối chuẩn một chiều. phân phối, $X \sim N(\mu_k, \sigma^2_k)$. Nhớ lại rằng hàm mật độ cho Phân phối chuẩn một chiều được cho trong (4.16). Chứng minh rằng trong Trong trường hợp này, bộ phân loại Bayes không phải là tuyến tính. Hãy lập luận rằng trên thực tế nó là tuyến tính. bậc hai. Gợi ý: Đối với bài toán này, bạn nên làm theo các lập luận đã được trình bày trong... Mục 4.4.1, nhưng không đưa ra giả định rằng $\sigma^2_1 = \dots = \sigma^2_K$.
- Khi số lượng đặc trưng p lớn, hiệu suất của KNN và các phương pháp cục bộ khác có xu hướng suy giảm. Thực hiện dự đoán chỉ sử dụng các quan sát gần với quan sát cần kiểm tra để đưa ra dự đoán. Hiện tượng này là được gọi là lỗi nguyên của chiều không gian, và nó liên quan đến thực tế là các phương pháp phi tham số thường hoạt động kém hiệu quả khi p lớn. Chúng ta Tôi sẽ tiến hành điều tra lỗi nguyên này.



lỗi nguyên
của chiều không gian

194 4. Phân loại

- (a) Giả sử ta có một tập hợp các quan sát, mỗi quan sát có các phép đo trên một đặc trưng X ($p = 1$). Ta giả sử X được phân bố đều trên khoảng $[0, 1]$. Mỗi quan sát đều có một giá trị phản hồi tương ứng. Giả sử ta muốn dự đoán phản hồi của một quan sát thử nghiệm chỉ sử dụng các quan sát nằm trong phạm vi 10% của khoảng giá trị X gần nhất với quan sát thử nghiệm đó. Ví dụ, để dự đoán phản hồi cho một quan sát thử nghiệm với $X = 0,6$, ta sẽ sử dụng các quan sát trong khoảng $[0,55, 0,65]$. Trung bình, ta sẽ sử dụng bao nhiêu phần trăm số quan sát có sẵn để thực hiện dự đoán?
- (b) Bây giờ giả sử chúng ta có một tập hợp các quan sát, mỗi quan sát có các phép đo trên $p = 2$ đặc trưng, X_1 và X_2 . Chúng ta giả sử rằng (X_1, X_2) được phân bố đều trên $[0, 1] \times [0, 1]$. Chúng ta muốn dự đoán phản hồi của một quan sát thử nghiệm chỉ sử dụng các quan sát nằm trong phạm vi 10% của phạm vi X_1 và trong phạm vi 10% của phạm vi X_2 gần nhất với quan sát thử nghiệm đó. Ví dụ, để dự đoán phản hồi cho một quan sát thử nghiệm với $X_1 = 0,6$ và $X_2 = 0,35$, chúng ta sẽ sử dụng các quan sát trong phạm vi $[0,55, 0,65]$ cho X_1 và trong phạm vi $[0,3, 0,4]$ cho X_2 . Trung bình, chúng ta sẽ sử dụng bao nhiêu phần trăm các quan sát có sẵn để thực hiện dự đoán?
- (c) Bây giờ giả sử chúng ta có một tập hợp các quan sát trên $p = 100$ đặc trưng. Một lần nữa, các quan sát được phân bố đều trên mỗi đặc trưng, và mỗi đặc trưng có giá trị từ 0 đến 1. Chúng ta muốn dự đoán phản hồi của một quan sát thử nghiệm bằng cách sử dụng các quan sát nằm trong phạm vi 10% của mỗi đặc trưng gần nhất với quan sát thử nghiệm đó. Chúng ta sẽ sử dụng bao nhiêu phần trăm số quan sát có sẵn để thực hiện dự đoán?
- (d) Dựa vào câu trả lời của bạn ở các phần (a)-(c), hãy lập luận rằng một nhược điểm của KNN khi p lớn là có rất ít quan sát huấn luyện "gần" bất kỳ quan sát kiểm tra nào.
- (e) Bây giờ giả sử chúng ta muốn đưa ra dự đoán cho một quan sát thử nghiệm bằng cách tạo ra một siêu lập phương p chiều có tâm là quan sát thử nghiệm đó, chứa trung bình 10% các quan sát huấn luyện. Với $p = 1, 2$ và 100 , độ dài mỗi cạnh của siêu lập phương là bao nhiêu? Hãy bình luận về câu trả lời của bạn.

Lưu ý: Hình lập phương đa chiều (hypercube) là sự tổng quát hóa của hình lập phương lên số chiều bất kỳ. Khi $p = 1$, hình lập phương đa chiều chỉ đơn giản là một đoạn thẳng, khi $p = 2$ nó là một hình vuông, và khi $p = 100$ nó là một hình lập phương 100 chiều.

5. Bây giờ chúng ta sẽ xem xét sự khác biệt giữa LDA và QDA.

- (a) Nếu ranh giới quyết định Bayes là tuyến tính, liệu chúng ta có kỳ vọng LDA hay QDA sẽ hoạt động tốt hơn trên tập huấn luyện? Trên tập kiểm tra?
- (b) Nếu ranh giới quyết định Bayes không tuyến tính, liệu chúng ta có kỳ vọng LDA hay QDA sẽ hoạt động tốt hơn trên tập huấn luyện? Trên tập kiểm tra?

(c) Nhìn chung, khi kích thước mẫu n tăng lên, liệu độ chính xác dự đoán của QDA so với LDA sẽ cải thiện, giảm sút hay không thay đổi? Tại sao?

(d) Đúng hay Sai: Ngay cả khi ranh giới quyết định Bayes cho một bài toán nhất định là tuyến tính, chúng ta có thể đạt được tỷ lệ lỗi kiểm thử tốt hơn bằng cách sử dụng QDA thay vì LDA vì QDA đủ linh hoạt để mô hình hóa ranh giới quyết định tuyến tính. Hãy giải thích câu trả lời của bạn.
trả lời.

6. Giả sử ta thu thập dữ liệu cho một nhóm sinh viên trong lớp thống kê với các biến X_1 = số giờ học, X_2 = điểm trung bình đại học (GPA), và Y = đạt điểm A. Ta sử dụng hồi quy logistic và thu được các hệ số ước lượng, $\beta^0 = -6$, $\beta^1 = 0,05$, $\beta^2 = 1$.

(a) Ước tính xác suất để một sinh viên học 40 giờ và có điểm trung bình đại học là 3.5 đạt điểm A trong môn học. (b) Sinh viên ở phần (a) cần học

bao nhiêu giờ để có 50% cơ hội đạt điểm A trong môn học?

7. Giả sử chúng ta muốn dự đoán liệu một cổ phiếu cụ thể có phát hành cổ tức trong năm nay hay không ("Có" hoặc "Không") dựa trên X , tỷ lệ lợi nhuận của năm trước. Chúng ta xem xét một số lượng lớn các công ty và phát hiện ra rằng giá trị trung bình của X đối với các công ty đã phát hành cổ tức là $X^+ = 10$, trong khi giá trị trung bình đối với các công ty không phát hành cổ tức là $X^- = 0$. Ngoài ra, phương sai của X đối với hai nhóm công ty này là $\sigma^2 = 36$. Cuối cùng, 80% các công ty đã phát hành cổ tức. Giả sử X tuân theo phân phối chuẩn, hãy dự đoán xác suất một công ty sẽ phát hành cổ tức trong năm nay khi tỷ lệ lợi nhuận của công ty đó là $X = 4$ vào năm trước.

năm.

Gợi ý: Hãy nhớ rằng hàm mật độ xác suất của biến ngẫu nhiên phân phối chuẩn là $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} (x - \mu)^2 / 2\sigma^2$. Bạn sẽ cần sử dụng định lý Bayes.

8. Giả sử ta lấy một tập dữ liệu, chia nó thành hai tập huấn luyện và kiểm tra có kích thước bằng nhau, rồi thử hai quy trình phân loại khác nhau.

Đầu tiên, chúng ta sử dụng hồi quy logistic và thu được tỷ lệ lỗi là 20% trên dữ liệu huấn luyện và 30% trên dữ liệu kiểm tra. Tiếp theo, chúng ta sử dụng phương pháp lân cận gần nhất (1-nearest neighbor - $K = 1$) và thu được tỷ lệ lỗi trung bình (trung bình trên cả tập dữ liệu huấn luyện và kiểm tra) là 18%. Dựa trên các kết quả này, chúng ta nên ưu tiên sử dụng phương pháp nào để phân loại các quan sát mới?

Tại sao?

9. Bài toán này liên quan đến xác suất.

(a) Trung bình, có bao nhiêu phần trăm người có xác suất vỡ nợ thể tín dụng là 0,37 sẽ thực sự vỡ nợ? (b) Giả sử một cá nhân có 16% khả năng vỡ nợ thể tín

dụng. Xác suất người đó vỡ nợ là bao nhiêu?

10. Phương trình 4.32 đã suy ra biểu thức cho $\log \Pr(Y=K|X=x)$ bằng $\frac{\Pr(Y=K|X=x)}{\text{cách đặt trong đó}}$ trong



$p > 1$, sao cho giá trị trung bình của lớp thứ k , μ_k , là một vectơ p chiều và ma trận hiệp phương sai chung Σ là ma trận $ap \times p$.

Tuy nhiên, trong trường hợp $p = 1$, (4.32) có dạng đơn giản hơn, vì các giá trị trung bình $\mu_1, \dots, \mu_K$ và phương sai σ^2 là các đại lượng vô hướng. Trong trường hợp đơn giản hơn này, hãy lặp lại phép tính trong (4.32) và cung cấp các biểu thức cho a_k và b_{kj} theo π_k , π_K , μ_k , μ_K và σ^2 .

lời của bạn 11. Hãy tìm ra các dạng chi tiết và b_{kj} trong (4.33). Câu trả lời của a_k , b_{kj} , nên bao gồm π_k , π_K , μ_k , μ_K , Σ_k và Σ_K .



12. Giả sử bạn muốn phân loại một quan sát X thành táo và cam. Bạn sử dụng mô hình hồi quy logistic và thấy rằng

$$\Pr(Y = \text{orange} | X = x) = \frac{\exp(\beta^0 + \beta^1 x)}{\exp(\beta^0 + \beta^1 x) + 1}$$

Bạn của bạn áp dụng mô hình hồi quy logistic cho cùng dữ liệu bằng cách sử dụng công thức softmax trong (4.13) và thấy rằng

$$\Pr(Y = \text{cam} | X = x) = \frac{\exp(\alpha^{\text{orange}}_0 + \alpha^{\text{orange}}_1 x)}{\exp(\alpha^{\text{orange}}_0 + \alpha^{\text{orange}}_1 x) + \exp(\alpha^{\text{apple}}_0 + \alpha^{\text{apple}}_1 x)}$$

- Tỷ lệ logarit xác suất của cam so với táo trong mô hình của bạn là bao nhiêu?
- Tỷ lệ logarit xác suất của cam so với táo trong mô hình của bạn bè bạn là bao nhiêu? người mẫu?
- Giả sử trong mô hình của bạn, $\beta^0 = 2$ và $\beta^1 = 1$. Các ước lượng hệ số trong mô hình của bạn bè bạn là gì? Hãy trình bày càng chi tiết càng tốt.
- Bây giờ giả sử bạn và bạn của bạn cùng áp dụng hai mô hình tương tự trên một tập dữ liệu khác. Lần này, bạn của bạn nhận được các ước lượng hệ số $\alpha^{\text{orange}}_0 = 1, 2$, $\alpha^{\text{orange}}_1 = -2$, $\alpha^{\text{orange}}_0 = 3$, $\alpha^{\text{orange}}_1 = 0, 6$. Các ước lượng hệ số trong mô hình của bạn là bao nhiêu? (e)

Cuối cùng, giả sử bạn áp dụng cả hai mô hình từ (d) cho một tập dữ liệu với 2.000 quan sát thử nghiệm. Bạn kỳ vọng nhãn lớp dự đoán từ mô hình của bạn sẽ trùng khớp với nhãn lớp từ mô hình của bạn bạn trong bao nhiêu phần trăm thời gian? Giải thích câu trả lời của bạn.

Đã áp dụng

13. Câu hỏi này cần được trả lời bằng cách sử dụng bộ dữ liệu Hàng tuần, một phần của gói ISLP. Bộ dữ liệu này có tính chất tương tự như dữ liệu Smarket từ bài thực hành của chương này, ngoại trừ việc nó chứa 1.089 lợi nhuận hàng tuần trong 21 năm, từ đầu năm 1990 đến cuối năm 2010.

- Trình bày một số bản tóm tắt bằng số liệu và đồ họa của Tuần Dữ liệu. Có vẻ như có bất kỳ quy luật nào không?

- (b) Sử dụng toàn bộ tập dữ liệu để thực hiện hồi quy logistic với **Hướng** là biến phản hồi và năm biến trễ cộng với **Khối lượng** là các biến dự báo. Sử dụng hàm tóm tắt để in kết quả. Có biến dự báo nào có vẻ có ý nghĩa thống kê không? Nếu có, đó là những biến nào?
- (c) Tính ma trận nhầm lẫn và tỷ lệ dự đoán chính xác tổng thể. Giải thích ma trận nhầm lẫn cho bạn biết điều gì về các loại lỗi mà hồi quy logistic mắc phải.
- (d) Bây giờ hãy xây dựng mô hình hồi quy logistic sử dụng dữ liệu huấn luyện từ năm 1990 đến năm 2008, với **Lag2** là biến dự đoán duy nhất. Tính ma trận nhầm lẫn và tỷ lệ dự đoán chính xác tổng thể cho dữ liệu được giữ lại (tức là dữ liệu từ năm 2009 và 2010).
- (e) Lặp lại (d) bằng cách sử dụng
- LDA. (f) Lặp lại (d) bằng cách sử dụng QDA. (g) Lặp lại (d) bằng cách sử dụng KNN với $K = 1$. (h) Lặp lại (d) bằng cách sử dụng
- Naive Bayes. (i) Phương pháp nào trong số này dường như cho kết quả tốt nhất về Dữ liệu này?
- (j) Thử nghiệm với các tổ hợp khác nhau của các biến dự đoán, bao gồm cả các phép biến đổi và tương tác có thể có, cho mỗi phương pháp. Báo cáo các biến, phương pháp và ma trận nhầm lẫn liên quan cho thấy kết quả tốt nhất trên dữ liệu được giữ lại. Lưu ý rằng bạn cũng nên thử nghiệm với các giá trị cho K trong bộ phân loại KNN.

14. Trong bài toán này, bạn sẽ phát triển một mô hình để dự đoán liệu một chiếc xe cụ thể có mức tiêu hao nhiên liệu cao hay thấp dựa trên tập dữ liệu **Auto**.

- (a) Tạo một biến nhị phân, **mpg01**, chứa giá trị 1 nếu **mpg** có giá trị lớn hơn giá trị trung vị và giá trị 0 nếu **mpg** có giá trị nhỏ hơn giá trị trung vị. Bạn có thể tính giá trị trung vị bằng phương thức **median()** của khung dữ liệu. Lưu ý rằng bạn có thể thấy hữu ích khi thêm cột **mpg01** vào khung dữ liệu bằng cách gán. Giả sử bạn đã lưu khung dữ liệu dưới dạng **Auto**, điều này có thể được thực hiện như sau:

```
Auto['mpg01'] = mpg01
```

- (b) Khám phá dữ liệu bằng đồ thị để điều tra mối liên hệ giữa **mpg01** và các đặc điểm khác. Đặc điểm nào trong số các đặc điểm khác có vẻ hữu ích nhất trong việc dự đoán **mpg01**? Biểu đồ phân tán và biểu đồ hộp có thể là những công cụ hữu ích để trả lời câu hỏi này. Mô tả kết quả của bạn.
- (c) Chia dữ liệu thành tập huấn luyện và tập kiểm tra. (d) Thực hiện LDA trên dữ liệu huấn luyện để dự đoán **mpg01** bằng cách sử dụng các biến có vẻ liên quan nhất đến **mpg01** trong (b). Sai số kiểm tra của mô hình thu được là bao nhiêu?

- (e) Thực hiện QDA trên dữ liệu huấn luyện để dự đoán `mpg01` bằng cách sử dụng các biến có vẻ liên quan nhất với `mpg01` trong (b). Sai số kiểm định của mô hình thu được là bao nhiêu? (f) Thực hiện hồi quy logistic trên dữ liệu huấn luyện để dự đoán `mpg01` bằng cách sử dụng các biến có vẻ liên quan nhất với `mpg01` trong (b). Sai số kiểm định của mô hình thu được là bao nhiêu? (g) Thực hiện Naive Bayes trên dữ liệu huấn luyện để dự đoán `mpg01` bằng cách sử dụng các biến có vẻ liên quan nhất với `mpg01` trong (b). Sai số kiểm định của mô hình thu được là bao nhiêu? (h) Thực hiện KNN trên dữ liệu huấn luyện, với một số giá trị của K, để dự đoán `mpg01`. Chỉ sử dụng các biến có vẻ liên quan nhất với `mpg01` trong (b). Bạn thu được sai số kiểm định nào? Giá trị K nào cho kết quả tốt nhất trên tập dữ liệu này?

15. Bài toán này liên quan đến việc viết hàm.

- (a) Viết một hàm, `Power()`, in ra kết quả của việc nâng 2 lên lũy thừa 3. Nói cách khác, hàm của bạn nên tính 2^3 và in ra kết quả.

Gợi ý: Hãy nhớ rằng `x**a` nâng `x` lên lũy thừa `a`. Sử dụng hàm `print()` để hiển thị kết quả. (b) Tạo một

hàm mới, `Power2()`, cho phép bạn truyền vào hai số bất kỳ, `x` và `a`, và in ra giá trị của `x**a`. Bạn có thể làm điều này bằng cách bắt đầu hàm của mình với dòng

```
def Power2(x, a):
```

Bạn có thể gọi hàm của mình bằng cách nhập, ví dụ:

```
Power2(3, 8)
```

trên dòng lệnh. Kết quả xuất ra sẽ là 38, cụ thể là 6.561.

- (c) Sử dụng hàm `Power2()` mà bạn vừa viết, hãy tính 103, 817 và 1313. (d) Bây giờ hãy tạo

một hàm mới, `Power3()`, hàm này thực sự trả về kết quả `x**a` dưới dạng một đối tượng Python, thay vì chỉ in nó ra màn hình. Nghĩa là, nếu bạn lưu trữ giá trị `x**a` trong một đối tượng có tên là `result` trong hàm của mình, thì bạn chỉ cần `trả về` kết quả này bằng dòng lệnh sau:

trở lại

```
trả về kết quả
```

Lưu ý rằng dòng trên phải là dòng cuối cùng trong hàm của bạn và phải được thụt vào 4 khoảng trắng.

- (e) Bây giờ, sử dụng hàm `Power3()`, hãy tạo đồ thị của $f(x) = x^2$. Trục `x` nên hiển thị một dãy số nguyên từ 1 đến 10, và trục `y` nên hiển thị x^2 . Hãy đặt nhãn cho các trục một cách thích hợp và sử dụng tiêu đề phù hợp cho hình. Cần nhắc hiển thị trục `x`, trục `y`, hoặc cả hai trên thang logarit. Bạn có thể thực hiện điều này bằng cách sử dụng các phương thức `ax.set_xscale()` và `ax.set_yscale()` của các trục mà bạn đang vẽ đồ thị.

`.set_xscale()` `.set_yscale()`

- (f) Tạo một hàm, `PlotPower()`, cho phép bạn tạo đồ thị của x theo $x**a$ với một giá trị a cố định và một dãy các giá trị của x . Ví dụ, nếu bạn gọi

```
PlotPower(np.arange(1, 11), 3)
```

Sau đó, cần tạo một đồ thị với trục x nhận các giá trị 1, 2, ..., 10 và trục y nhận các giá trị 13, 23, ..., 103.

16. Sử dụng bộ dữ liệu `Boston`, hãy xây dựng các mô hình phân loại để dự đoán xem một vùng ngoại ô cụ thể có tỷ lệ tội phạm cao hơn hay thấp hơn mức trung bình. Khám phá các mô hình hồi quy logistic, LDA, Naive Bayes và KNN sử dụng các tập con khác nhau của các biến dự đoán. Mô tả các phát hiện của bạn.

Gợi ý: Bạn sẽ phải tự tạo biến phản hồi bằng cách sử dụng các biến có sẵn trong tập dữ liệu `Boston`.